



COMP390

2025/26

Simulated Automotive Software (SAS)

Steven Beardsley
201631843

BSc Computer Science with a Year in Industry

SCHOOL OF
COMPUTER SCIENCE & INFORMATICS

University of Liverpool
L69 3DR

Dedicated to Anna, Lena, Ed and Joe.

Acknowledgements

I'd like to thank my parents for their support through my studies, and my supervisor, Valentina Tamma, for supporting me throughout this project.



COMP390

2025/26

Simulated Automotive Software (SAS)

Steven Beardsley

201631843

BSc Computer Science with a Year in Industry

SCHOOL OF
COMPUTER SCIENCE & INFORMATICS

University of Liverpool
L69 3DR

Abstract

This dissertation presents the design and implementation of Simulated Automotive Software (SAS), a modular digital twin of an automotive system built on industry-standard languages, protocols and tooling. The project comprises two systems: NEON, a collection of independently running Electronic Control Unit (ECU) simulations containerised using Docker, and the Vehicle Simulation Platform (VSP), a Windows desktop application used to configure, deploy and monitor the simulation in real time.

NEON consists of three containerised C++ components: an Engine Control Module, a Transmission Control Module, and a Dashboard. These are networked together using an adapted implementation of the Controller Area Network (CAN) protocol, serialised at the bit level and transmitted over UDP. The Engine Control Module simulates torque generation, fuel consumption, and thermal dynamics using physically grounded mathematical models. The Dashboard bridges the NEON network and the VSP through the use of HTTP and WebSockets, while also routing commands inbound from the VSP as CAN messages.

The VSP is implemented using the .NET framework following a Model-View-Controller architecture, providing real-time telemetry graphs and vehicle control inputs, allowing the user to configure, control and monitor the NEON system.

Statement of Ethical Compliance

Data Category: A

Participant Category: 0

I confirm that I have followed the ethical guidelines during this project. Further details can be found in the relevant sections of this dissertation.

Table of Contents

Acknowledgements	3
Abstract	5
Statement of Ethical Compliance	5
Table of Contents	6
Table of Figures	8
Glossary	9
1 Introduction & Background	10
1.1 Project Aim	10
1.2 Project Motivation	10
1.3 Project Overview	11
2 Requirements Analysis	11
2.1 Project Objectives	11
2.2 Software Requirements	11
3 Design & Implementation	12
3.1 NEON	12
3.1.1 System Architecture	12
3.1.2 Simulated ECU Components	13
3.1.3 Libraries	20
3.1.4 Software Development Tools	26
3.1.5 External Libraries	29
3.1.6 Coding conventions	30
3.2 Vehicle Simulation Platform	33
3.2.1 Graphical User Interface Designs	33
3.2.2 Architecture	36
3.2.3 .NET Framework	37
3.3 UML Interaction Diagrams	38
3.3.1 Configuration and Deployment	38
3.3.2 Gear Change Request Event	39
3.3.3 Car Stalling Event	40
4 Testing	41
4.1 Unit Tests	41
4.2 Acceptance Tests	42
4.3 Static Analysis	43
4.4 Simulation Analysis	43

5 Evaluation	47
5.1 Achievements of Requirements.....	47
5.2 Technical Strengths.....	48
5.3 Limitations and Challenges	49
5.4 Comparison to Industry Systems	49
6 Project Ethics	50
7 Conclusion & Future Work	50
8 BCS Criteria & Self-Reflection	51
9 References	54
10 Appendices	55
Appendix A – Project Planning.....	56
Appendix B – System Verification	56
Appendix C – Communication Specification	58
Appendix D – Configuration.....	61
Appendix E – Implementation Code	62

Table of Figures

Figure 1 System architecture diagram illustrating the different components of the SAS	13
Figure 2 Torque calculations within the Engine class	15
Figure 3 Code snippet showing the hard limit for the lower bound of engine temperature .	18
Figure 4 Code snippet from the ThermalModel class that shows the hysteresis based approach to prevent overheating.....	18
Figure 5 handleDatagram method definition	22
Figure 6 The class declaration for the ICANMessage class	23
Figure 7 Safe accessor methods to access the configuration values.....	25
Figure 8 An example log line output from a NEON container	26
Figure 9 Example of the relationship between include guards and namespaces	31
Figure 10 CMake file structure for the NEON project	32
Figure 11 Example class, highlighting the naming conventions of member functions and attributes.....	33
Figure 12 The VSP Deploy Page after successfully deploying the NEON system	34
Figure 13 The Track Page on the VSP.....	35
Figure 14 Screenshot of the logging page	35
Figure 15 Screenshot of the NEON Live Telemetry Window	36
Figure 16 UML Sequence diagram illustrating the deployment process	38
Figure 17 UML Sequence diagram displaying a gear change request	39
Figure 18 UML Sequence diagram showing the engine stalled event	40
Figure 19 NEON unit test report generated by a Python script	42
Figure 20 Snapshot of the NEON telemetry graphs responding to gear changes.....	44
Figure 21 NEON telemetry graphs when accelerating from stationary	45
Figure 22 Snapshot showing the hysteresis model for engine temperature	45
Figure 23 Telemetry graph displaying engine braking behaviour	46
Figure 24 Telemetry graph displaying brake input behaviour.....	47

Glossary

Term	Definition
Controller Area Network (CAN)	A layer 2 networking protocol that is the industry standard for how different sub-systems of an automotive communicate.
Digital twin	Digital twins are virtual representations of a physical object or system that uses real-time data to accurately reflect its real-world counterpart's behaviour, performance and condition (Gallagher, 2020)
Electronic Control Unit (ECU)	A sub-system of an automotive that includes a management aspect and the device it is controlling.
Engine Control Module (ECM)	A module which controls the engine, it's fuel, thermal, drive and torque dynamics.
HyperText Transfer Protocol (HTTP)	An application-layer protocol used for transmitting data over the web.
Interface Control Document (ICD)	A document which defines the API, and message definitions sent across the network.
JavaScript Object Notation (JSON)	A lightweight, text-based data format used to store and exchange structured data in a way that is easy for humans to read and machines to parse.
Networked ECU Orchestration Nodes (NEON)	The culmination of networked ECU simulations which form to become an automotive simulation.
Simulated Automotive Software (SAS)	The complete project, comprising NEON and the VSP systems.
Transmission Control Module (TCM)	A module which controls the transmission system with NEON.
Vehicle Simulation Platform (VSP)	A Windows desktop application that is used to configure, deploy and control the NEON system.
WebSockets	A full-duplex communication protocol that enables persistent, real-time, bidirectional data exchange between a client and server over a single TCP connection.

1 Introduction & Background

Simulated Automotive Software (SAS) provides a test bed for new, interoperable, automotive components to be developed and tested within a virtual, synthetic environment. The software employs modern digital twin philosophies, and bespoke simulations to create a full end-to-end system.

1.1 Project Aim

The aim of SAS is to create a test bed for new automotive components to be tested and deployed in a completely virtual, controlled and synthetic environment.

As part of this initial aim, the project has been built mirroring industry standard conventions, standards, techniques and languages, some of which I have picked up during my industrial placement. This was the motivating factor for a containerised approach to the automotive simulation, deploying it onto a much more lightweight and portable Linux Operating System. As part of the development of the automotive digital twin, the project aims to effectively simulate how the different ECU components in an automotive communicate, to add to the completeness and correctness of my simulation. To do this, the simulated components were made to communicate over a Controller Area Network (CAN) protocol, adapted over UDP.

In order to configure, deploy, control and monitor the automotive simulation, a Graphical User Interface (GUI) is also included within the project scope, named the Vehicle Simulation Platform (VSP). This is a Windows desktop application, interfacing with the automotive simulation and presenting its feedback data in real-time.

1.2 Project Motivation

The motivation behind this project stems from the increasing adoption of digital twin methodologies within modern engineering (Prime Nearshore, n.d.).

Digital twins are digital replicas of actual or potential physical devices, processes, people, places, and systems that can be used to run simulations (IOWN, n.d.). These digital twins allow complex and expensive hardware to be modelled, tested and iterated upon within low-cost, controlled synthetic environments. I saw this first-hand during my industrial placement, where I observed the application of digital twins in the context of aerospace communication systems, where they were used to simulate and validate components of a jet aircraft, often integrated with real hardware. This highlighted the potential of digital twins when accelerating development cycles, improving testing and reproducibility, and enhancing system robustness.

The aim of SAS is to create a microcosm of something that a larger automotive engineering company might create, with this in mind, I have mirrored the industry technical stacks and employed industry standard languages, and practices at each stage of development during this project.

1.3 Project Overview

SAS is comprised of two systems: the automotive simulation, NEON, and the graphical user interface, the VSP.

1.3.1 Networked ECU Orchestration Nodes

Networked ECU Orchestration Nodes or NEON, is the collection of networked modular components that form the automotive simulation.

For the context of SAS, NEON is comprised of three components to demonstrate the feasibility, architecture and potential of NEON. Acting as the beginnings of creating a much larger complex test bed, building on the architecture and framework which the 3 subsystems, called Electronic Control Units, or ECUs, will provide.

These ECUs are the:

- Dashboard, which acts as the primary interface between the user interface and the NEON system.
- Engine Control Module (ECM)
- Transmission Control Module (TCM)

These programs have been developed as C++ binaries, compiled and ran on Linux Ubuntu, and containerised using Docker, deployed on Docker Desktop. These subsystems are networked together using an adapted version of the industry standard, layer 2 CAN protocol.

1.3.2 Vehicle Simulation Platform

The Vehicle Simulation Platform, or VSP, is a Windows desktop application using the .NET framework, capitalising on architectures such as Model-View-Controller to create an intuitive, multi-threaded graphical user interface.

The primary aim for the VSP is to control, monitor, and test the NEON system.

2 Requirements Analysis

This section of the report will look at the software aims and requirements, these are the targets and baselines for this project.

2.1 Project Objectives

My project objectives, as outlined in the initial proposal, are as follows:

- To be a professional looking desktop application that allows for the configuration, deployment and monitoring of a simulated vehicle.
- To simulate the behaviours of multiple containerised ECUs.
- To implement the industry standard CAN networking protocol to communicate the different ECUs.

These objectives collectively establish the foundation for SAS, a modular, scalable and realistic automotive simulation platform.

2.2 Software Requirements

The functional and non-functional requirements of the system are defined below:

1. Graphical User Interface (GUI)

The system shall provide an intuitive graphical user interface to enable users to monitor and control the NEON system.

- 1.1. The GUI shall be implemented as a Windows desktop application and be accessible via the Windows Start Menu.
- 1.2. The GUI shall be designed with usability in mind, ensuring it is intuitive and user-friendly.
- 1.3. The GUI shall support the configuration and deployment of the NEON system.
- 1.4. The GUI shall display telemetry data in real time to reflect the current system state.

2. Automotive System Simulation

The system shall simulate key components of an automotive environment.

- 2.1. The system shall simulate an engine component, including fuel consumption, power output, and thermal dynamics.
- 2.2. The system shall simulate a Transmission Control Module (TCM).

3. System Architecture

The system shall adopt a containerised architecture to support modularity and future integration with physical hardware or external systems.

4. Real-Time Operation

The system shall operate with real-time or near real-time performance characteristics to ensure accurate simulation behaviour and responsive user interaction.

5. Communication Protocol

The system shall simulate the Controller Area Network (CAN) protocol for communication between components.

- 5.1. CAN communication shall be adapted over UDP to remove hardware dependencies while preserving message structure and behaviour.
- 5.2. The system shall support multiple message types and data formats for communication across the network.

3 Design & Implementation

This section describes the design and implementation of the two systems of SAS including how they have been developed, the communication protocols implemented and UML sequence diagrams illustrating the behaviours of the complete SAS system.

3.1 NEON

3.1.1 System Architecture

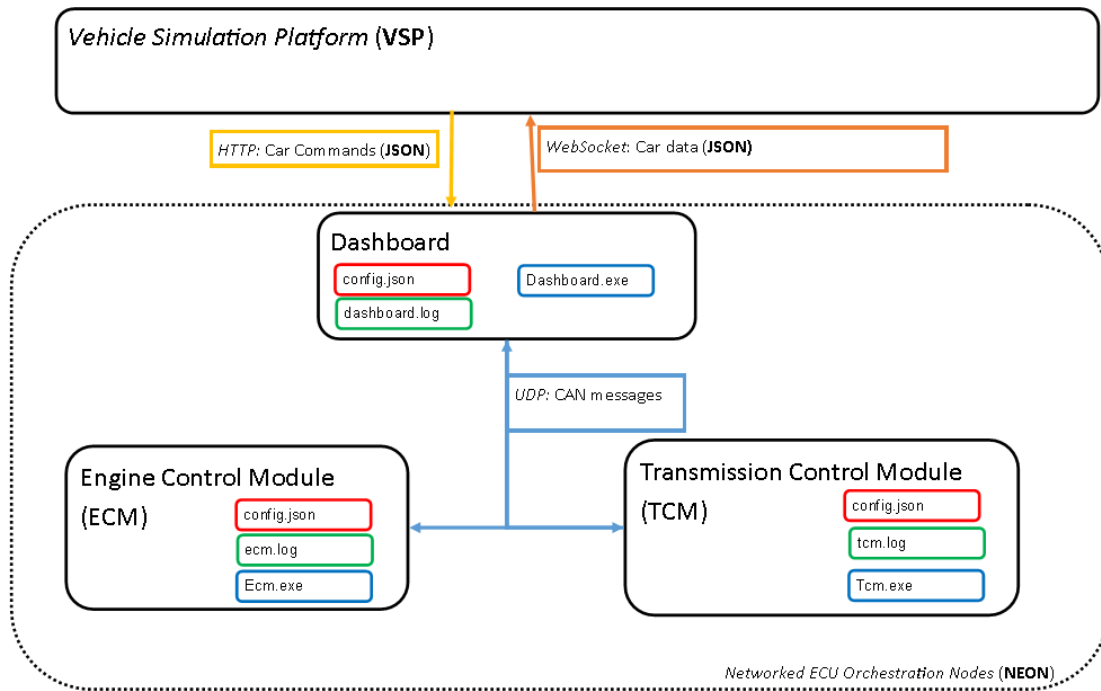


Figure 1 System architecture diagram illustrating the different components of the SAS

As shown in Figure 1, the system implements a range of components and networking protocols in order to communicate with the different parts of SAS. These will be covered in the later sections of this document.

3.1.2 Simulated ECU Components

To effectively simulate the movements and properties which culminate to form an automotive system, NEON simulates Electronic Control Units (ECUs). As mentioned, these are C++ binaries deployed within Docker containers, ran locally using Docker Desktop. Each container, as shown in Figure 1, contains a C++ binary, a log file, and a shared vehicle configuration file, generated by the VSP when the vehicle is configured on deployment.

3.1.2.1 Engine Control Module (ECM)

Subscribed CAN Messages:

- *CurrentGear (Status)*
- *Refuel (Control)*
- *ThrottleRequest (Control)*
- *BrakeRequest (Control)*

Output CAN Messages:

- *RPM (Status)*
- *Speed (Status)*
- *EngineTemperature (Status)*
- *LowFuel (Error)*
- *NoFuel (Error)*
- *EngineStalled (Error)*
- *ShiftUpRecommended (Error)*
- *ShiftDownRecommended (Error)*
- *EngineOverheating (Error)*

The Engine Control Module (ECM) is made up of two CMake targets, the ECM and the Engine.

The ECM target role is to interface between the engine simulation and the networking of NEON. As part of this it sets up the ECM within the network, adding the correct peers and utilising buffers to handle incoming and outgoing CAN messages, additionally, it also creates the container specific log file utilising the *LogFile* library mentioned in section 3.1.3.4.

The Engine target, *see appendix E.1 for the source code*, simulates the functions of an automotive engine. It also loads and applies the engine profile derived from the configuration file, containing attributes such as engine size and idle RPM etc. Some of the engine functionality has been abstracted into separate models, such as the fuel modelling and the thermal dynamics which contain more computation. This abstraction helps maintain encapsulation, code maintainability and portability, strengthening the containerised modular approach of this entire project.

The engine operates on a continuous simulation loop, governed by the *runLoop* and *step* functions. It utilises a steady clock to calculate a precise delta time, ensuring the physics remains consistent regardless of network jitters or processing overhead, ensuring a robust real-time solution. As mentioned, the *Engine* class also manages several interdependent subsystems including a fuel model that tracks displacement-based consumption, a thermal model for coolant temperature, and a transmission model that handles gear ratios and final drive calculations.

The simulation distinguishes between two primary mechanical states: *Neutral* and *InGear*. In neutral, the engine RPM is decoupled from the vehicle speed and is instead simulated as a first-order lag system responding to throttle input. When a gear is engaged, the RPM becomes a function of the wheel speed and the total gear reduction ratio. A critical behavioral feature is the stall logic, where the engine is flagged as stalled if the RPM exceeds the redline by a 10% margin, this simulates catastrophic mechanical failure.

Torque Generation

Within the engine simulation, torque curve generation is calculated dynamically using a Gaussian-like bell curve function:

$$\tau(rpm) = \tau_{max} \times e^{-0.5x^2}$$

Where x is a normalised factor representing how far the current RPM is from the “Peak Torque RPM” (defined as 60% of the usable rev range), and is calculated as:

$$x = \frac{rpm - rpm_{peak}}{spread}$$

This ensures that the torque rises from idle, peaks, and then naturally falls off as it approaches redline, accurately mimicking the behaviours of a real engine.

```
double Engine::torqueAtRpmNm(const double rpm) const
{
    constexpr double kMinTorqueNm = 1.0;
    constexpr double kPeakRpmFraction = 0.60; // peak torque at 60% of
the RPM range
    constexpr double kSpreadFraction = 0.22; // gaussian width as
fraction of RPM range
    constexpr double kMinTorqueFraction = 0.25; // torque floor as
fraction of peak

    const auto maxT = std::max(kMinTorqueNm, m_engineCfg.max_torque_nm);
    const auto redline = m_engineCfg.max_rpm;
    const auto idle = m_engineCfg.idle_rpm;

    if (redline <= idle + 1.0)
        return maxT;

    const auto rpmRange = redline - idle;
    const auto peakRpm = idle + kPeakRpmFraction * rpmRange;
    const auto spread = std::max(1.0, kSpreadFraction * rpmRange);
    const auto x = (rpm - peakRpm) / spread;

    return std::max(maxT * std::exp(-0.5 * x * x),
        MinTorqueFraction * maxT);
}
```

Figure 2 Torque calculations within the Engine class

The *torqueAtRpmNm* function, shown in Figure 2, returns the maximum available engine torque in Newton-metres at a given RPM. It models a realistic torque curve by placing the peak at 60% of the usable rev range, between idle and redline, and tapering off in both directions using a Gaussian bell curve shape. If the RPM range is too narrow to be meaningful, it simply returns the engine's maximum torque flat. Torque is also prevented from dropping below 25% of the maximum at any point, ensuring the engine always produces a minimum amount of torque across the entire rev range.

Drive Forces

When the car is in gear, the force propelling the vehicle is calculated by transferring the generated engine torque through the drivetrain to the wheels. The tractive force (F_{drive}) at the wheels is calculated as:

$$F_{drive} = \frac{\tau_{engine} \times R_{gear} \times R_{final} \times \eta_{drivetrain}}{r_{wheel}}$$

Where:

- τ_{engine} is the current engine output in Nm.
- R_{gear} is the selected gear ratio.
- R_{final} is the final drive train ratio.

- $\eta_{drivetrain}$ is the mechanical efficiency of the drive train.
- r_{wheel} is the radius of the wheel in metres

To determine the vehicle's acceleration, this drive force is opposed by the resistive forces (F_{res}) which include rolling resistance and aerodynamic drag:

$$F_{res} = C_{rolling} + (C_{drag} \times v^2)$$

The net force acting on the vehicle is therefore the drive force minus the resistive forces and any applied braking force. This net force is divided by the vehicle's mass to calculate the instantaneous acceleration via Newton's Second Law (Alexander, 2017).

Fuel Model

The *FuelTank* is modelled as a simulation of fuel storage and consumption within the engine simulation, see *appendix E.2 for the source code*, forming an important part of the powertrain behaviour. Its primary purpose is to track the quantity of fuel available to the engine over time and to simulate how that fuel is consumed under varying factors such as engine speed and throttle input.

I have chosen to encapsulate this behaviour within a dedicated class to maintain a clear separation within the engine and improve abstractions and code maintainability. This abstraction and encapsulation also lends itself well if in the future of the development, the fuel tank was made into a separate component, similar to the TCM.

The class is initialised with values derived from the configuration files, see an example configuration file in *appendix D.1*, found within the Docker container, such as the total tank capacity from the engine displacement, this introduces a simple but realistic scaling relationship between engine size and fuel storage. This relationship is implemented using a linear approximation, where capacity is calculated as:

$$capacity = 40.0 + (7.5 \times Displacement)$$

This results in larger engines being paired with larger fuel tanks (e.g., a 1.0L engine yields a 47.5L tank, while a 2.0L engine yields a 55L tank). The tank is initially set to full capacity, ensuring a consistent starting state for the simulation.

The core behaviour of the class is implemented in the *update* method, which models fuel consumption as a function of engine operating conditions over a discrete time loop. The fuel burn rate is calculated as:

$$burnRate = Base_{idle} + Load_{throttle} + R_{influence}$$

Where

- $Base_{idle}$ is the baseline idle consumption, set to $0.015 \times Displacement$
- $Load_{throttle}$ is the throttle contribution representing driver demand set to $0.015 \times Displacement \times Throttle$

- $RPM_{influence}$ represents the RPM-dependent load, set to $0.25 \times Displacement \times RPM_{norm} \times Throttle$ where RPM_{norm} is the current RPM normalised between idle and redline (0.0 to 1.0)

Additionally, the simulation models a rich mixture scenario under heavy load. If the engine approaches its redline (specifically, when $RPM_{norm} > 0.85$), the overall fuel burn rate is multiplied by a factor of 1.15 to simulate the engine dumping extra fuel to prevent detonation and manage cylinder temperatures.

To maintain robustness, the implementation includes safeguards such as clamping throttle input, preventing negative fuel levels, and early exiting when the tank is empty. Utility methods provide access to the current fuel level in both absolute (litres) and relative (percentage) terms, while additional functions support refuelling and incremental fuel addition. Collectively, these features allow the *FuelTank* class to integrate seamlessly with the wider engine model, contributing to a more realistic and responsive simulation by ensuring that fuel availability directly influences engine behaviour over time.

Temperature Model

The *ThermalModel* class simulates the thermal behaviour of the engine, tracking how its temperature evolves over time as a result of heat generation and cooling. The model is designed to reflect two competing physical processes; the heat produced by a running engine (heat generation), and the natural tendency of that heat to dissipate to the surrounding environment (cooling).

The model is initialised with a cold engine at ambient temperature, representing a standard cold-start condition. From that point, temperature is updated continuously based on engine speed, throttle position, and elapsed time.

Heat Generation

Before the heat can be calculated, engine speed is normalised into a load value $L \in [0,1]$ relative to the engine's idle and redline limits, clamped to ensure numerical stability:

$$L = clamp\left(\frac{w - w_{idle}}{w_{red} - w_{idle}}, 0, 1\right)$$

Heat is then produced through two distinct mechanisms. The first is friction: even with no throttle input, the mechanical load of spinning engine components generates heat that grows sharply as the engine approaches its redline. The second is combustion: when both throttle and engine speed are present together, significant additional heat is produced. These two contributions are weighted and summed to give a total rate of heat input:

$$Q_{in} = k_h(0.3L^2 + 0.7L\theta)$$

Where k_h is the heat rate coefficient and $\theta \in [0,1]$ is the throttle position. The quadratic friction term (L^2) means friction heat rises steeply near redline even with no throttle, while the combustion term ($L \times \theta$) requires both engine speed and throttle to be present. This split reflects the real-world behaviour where an engine spinning freely at high revs still gets hot, but sustained power delivery under load generates the most heat.

Cooling

Cooling is modelled as an exponential decay towards ambient temperature. This means the rate of cooling is proportional to how far above the ambient the engine currently sits, a very hot engine cools quickly at first, then more slowly as it approaches the equilibrium, mimicking real physical systems.

Heating and cooling are applied sequentially each timestep, heat is added first, then exponential cooling is applied to the result:

$$T_{new} = T_{amb} + (T + Q_{in}\Delta t - T_{amb})e^{-k_e\Delta t}$$

Where k_e is the cooling rate constant. A hard lower bound ensures the engine temperature can never fall below the ambient temperature, as seen below in Figure 3.

```
m_tempC = std::max(m_tempC, m_ambientC);
```

Figure 3 Code snippet showing the hard limit for the lower bound of engine temperature

Overheating Detection

```
if (m_tempC >= m_overheatC)
{
    m_overheating = true;
}
else if (m_tempC < m_overheatC - 5.0)
{
    m_overheating = false;
}
```

Figure 4 Code snippet from the ThermalModel class that shows the hysteresis based approach to prevent overheating

To enhance realism and robustness, the model includes a hysteresis-based overheating check. Using a hysteresis-based approach means that there is a gap between the activation and deactivation thresholds, ensuring that the engine overheating state does not immediately reverse when the monitored value fluctuates, this can be seen implemented within the code above in Figure 4. Once the temperature exceeds a threshold, a flag is set, and it remains active until the temperature falls sufficiently below that threshold ($m_overheatC$).

3.1.2.2 Transmission Control Module (TCM)

Subscribed CAN Messages:

- GearUpRequest (Control)
- GearDownRequest (Control)

Output CAN Messages:

- CurrentGear (Status)

The Transmission Control Module (TCM) is formed of two CMake targets, similar to the ECM, a *TCM* target which handles implementations such as threads, networks and logging, and the *Transmission*, which provides a thread safe implementation allowing for gears to be changed appropriately and accordingly.

The *TCM* class runs multiple threads. One of which sets up a thread-safe message inbox, comprised of a shared queue protected by a mutex and a condition variable, acting as a buffer for incoming CAN messages. A dedicated receiver thread continuously listens for raw byte payloads arriving on a specified UDP port (15000) and places them into this queue. A second thread, the process thread, blocks on the queue and is alerted whenever a new message arrives. It utilises the *BitReader* (see 3.1.3.2) class to inspect the message headers, identifying the message category and type. If the message is a control command requesting a gear shift upward or downward, it delegates directly to the *Transmission* object to perform the change.

Simultaneously, a third thread periodically broadcasts the transmission's current gear state around the network onto the CAN bus every 50ms, publishing a status message that peer modules (the ECM and Dashboard) are able to receive and act upon.

3.1.2.3 Dashboard

The dashboard has multiple roles within the NEON system.

One role is that of interfacing between the NEON and VSP systems. The dashboard collects and subscribes to multiple status messages before parsing them into a JSON format and posting them via the WebSocket protocol to the VSP. The WebSocket protocol was chosen over a standard HTTP connection as it doesn't rely on constant polling and works better for continuous constantly updating data, making it ideal for the constantly changing data of the car, with attributes such as speed and RPMs changing frequently. To fulfill this role, the dashboard maintains a CAN receiver thread and an associated message inbox, using the same mechanisms as the other containers (mentioned above in 3.1.2.2). The Dashboard also maintains a *DashboardDataSource* which is a JSON string of the data posted to the VSP, an example JSON can be seen in *appendix C.2*. This is sent to the VSP at a fixed interval of 50 milliseconds, providing the VSP with a near-real-time stream of vehicle telemetry.

Another role of the dashboard program is to parse the commands from the VSP into CAN messages to be sent around the network. Rather than commands such as 'change gear' being sent directly to containers, they are routed through the Dashboard, which converts them into CAN messages, defined in the ICD, see *appendix C.1*. This was a conscious design decision to create a better simulation of how CAN works within physical hardware inside automobiles. To achieve this goal, ports are exposed on the dashboard container and an HTTP server is hosted within the program on port 8081 via a *CommandHttpServer* instance running on its own dedicated thread, this relationship is shown in Figure 1. HTTP was chosen over WebSockets here, as the commands received are singular and not continuous, making it an ideal and relatively simple dynamic implementation. Within the code, these HTTP commands are parsed as strings before their values are extracted and converted into the correct CAN messages, which are then broadcast across the network.

As with the other NEON ECUs, the dashboard registers signal handlers for *SIGINT* and *SIGTERM* to support graceful shutdown. When a termination signal is received, the global running flag is set to false and the WebSocket acceptor is closed immediately, unblocking the main accept loop. The CAN receiver is then stopped, and all threads, including the CAN receiver, CAN processor, command server, and internal dashboard process thread, are joined in sequence before the process exits cleanly.

3.1.3 Libraries

This section of the report covers the libraries developed as part of the NEON system. These have been written in C++17, and aim to improve abstraction, code maintainability and encapsulation. These libraries have been developed with the aim to be lightweight, in some cases header-only, and easy to use, with clear functions and use cases. These libraries have all been included in the unit test framework, and as such, are subject to rigorous testing across extensive test cases.

3.1.3.1 Can

Contribution and Context

Library to simulate the layer 2 Controller Area Network (CAN) networking protocol (Corrigan, 2016). Simulates components such as the CAN bus, including broadcast functionality, and a receiver class to receive and parse CAN packets.

CAN is a layer 2 networking protocol, operating using a publish-subscribe model. Each component broadcasts its messages across the network on the CAN bus, and subscribe to messages which concern that respective subsystem, these messages are described within section 3.2.1.

As mentioned, I chose to use CAN to network the NEON components as this is the industry standard, implemented in most automobiles. I also chose to virtualise the protocol, sending my CAN data over UDP (J, 1980), this was to keep the project in scope, follow the digital twin ethos and make my simulation environment more deterministic by eliminating hardware dependencies.

Implementation

Bus

As part of the CAN library development a *Bus* class provides a lightweight abstraction of a CAN bus within the NEON simulation, enabling communication between each of the distributed components. Its primary aim is to emulate the broadcast style messaging behaviour of a real automotive CAN bus using UDP networking, allowing independently running processes, deployed within Docker containers, to exchange data in a loosely coupled manner.

Implementation wise, the class is built on top of Boost.Asio and manages a UDP socket bound to a specific port. During initialisation, the socket is configured with options such as address reuse and broadcast capability ensuring flexibility in containerised environments where services may be restarted or redeployed. This is especially important when I am testing my code, and components may be disconnected or restarted.

An *addPeer* function is responsible for dynamically resolving and registering other nodes on the network, using DNS resolution with retry logic to handle cases where dependent services are not yet available at startup. This is particularly important in the distributed Docker setup, where container initialisation order is not guaranteed. Successfully resolved peers are then stored in a thread-safe collection, allowing concurrent access from multiple parts of the ECU.

A *send* method implements the core communication mechanism, taking a serialised datagram and transmitting it to all registered peers, effectively mimicking the broadcast nature of a CAN bus. Thread safety is maintained through the use of the mutexes to protect both the send operation and the peer list, ensuring reliable operation in multi-threaded contexts.

Logging is also implemented within the CAN bus to aid debugging and traceability of message flow between components.

Overall, the *Bus* class encapsulates both the low-level networking details and the higher-level design goal of decoupled, message-driven communication.

Receiver

On the flip-side, the *Receiver* class complements the *Bus* component by providing the inbound communication pathway for the incoming CAN packets, enabling NEON components to receive data from other components in a reliable and thread safe manner. While the *Bus* class is responsible for broadcasting messages, the *Receiver* acts as a dedicated listener that continuously monitors the network for incoming datagrams and feeds them into the local processing pipeline.

The *Receiver* class is also built on Boost.Asio (boost, n.d.) and operates a UDP socket bound to a specified listening port, allowing it to receive messages from any peer on the Docker network. It is designed to run within its own execution thread, controlled by a shared atomic boolean flag *_running*, that enables safe start-up and shutdown behaviour.

```

void Receiver::handleDatagram(const uint8_t* data, std::size_t n, const
                             udp::endpoint& sender) const
{
    if (n == 0) return; // Ignore empty packets
    {
        std::vector<std::uint8_t> rawData(data, data + n);
        std::lock_guard<std::mutex> lock(inboxMutex_);
        inbox_.push(std::move(rawData));
    }

    inboxCv_.notify_one();
}

```

Figure 5 *handleDatagram* method definition

Packets are processed within the *handleDatagram* method, shown in Figure 5. This method performs minimal processing by copying the raw byte data into a vector and pushing it into a shared inbox queue, which is protected by a mutex to ensure thread safety. A condition variable is then notified to signal to other parts of the system that new data is available for processing.

The class design reflects a producer-consumer model, where the *Receiver* acts as the producer of incoming network messages and other system components act as consumers that interpret and process these messages asynchronously. This decoupling ensures that the system avoids blocking execution paths and improves overall responsiveness. Additionally, the use of simple raw datagram forwarding keeps the communication layer generic and flexible, allowing higher-level protocols or message formats to be implemented independently.

Message Categories

There are 3 message categories across the NEON network:

- *Control*, to edit, change or manage a part of NEON
- *Status*, broadcasting the current state of a value
- *Error*, to post a warning or error across the network

These are stored in an enum class within the *can* namespace, and indicate the type of message. These help simplify the switch-case statements when each component is deciding if the message is something that it subscribes to.

Message Types

The message types are defined within the NEON ICD, *see appendix C.1* for a detailed overview of each message.

Messages

```
#ifndef SHARED_CAN_MESSAGE_ICANMESSAGE_H
#define SHARED_CAN_MESSAGE_ICANMESSAGE_H

#include <vector>
#include <cstdint>

namespace shared::can::message
{
    class ICanMessage
    {
    protected:
        std::vector<std::uint8_t> m_buffer;

    public:
        virtual ~ICanMessage() = default;

        [[nodiscard]]
        const std::vector<std::uint8_t>& getRawData() const { return m_buffer; }

        explicit operator const std::vector<std::uint8_t>& () const
        {
            return m_buffer;
        }
    };
}

#endif
```

Figure 6 The class declaration for the *ICanMessage* class

Multiple message classes have been created, and defined within the ICD, *appendix C.1*. Each message class inherits from the interface *ICanMessage*, shown in Figure 6, which serves as a foundation for high-level message constructs. This helps ensure that all messages conform to a consistent structure before being passed to the communication layer. By standardising how the data is represented and accessed, this interface helps ensure interoperability between the components whilst simplifying the communications.

The *ICanMessage* class shown in Figure 6 encapsulates a raw byte buffer which represents the serialised form of the CAN message. This buffer is the exact payload transmitted over the network by the *Bus*.

3.1.3.2 Bit Parser

Contribution

The bit parser library provides a lightweight, deterministic mechanism for serialising and deserialising structured data at the bit level, forming a core part of the NEON communication stack. It is designed to support compact, CAN-style message encoding, where fields may not align cleanly to byte boundaries and strict control over bit layout is

required. It is a header only library, making static linking quicker and easier to implement across the codebase.

Implementation

The implementation is centred around two primary classes: *BitReader* and *BitWriter*. These operate over a shared abstraction of contiguous memory using the *Span* and *ByteBuffer* utility classes, ensuring efficient, zero-copy access where possible. The system is further parameterised by configurable *BitOrder* and *ByteOrder*, allowing the library to flexibly support both MSB-first and LSB-first bit packing, as well as big endian and little endian byte layouts. This is particularly important for interoperability with different network protocols and embedded system conventions. By implementing these configurable options, I extend my system's potential of being integrated with real hardware, which may use different endianness and bit packing orders. This goes back to my software aim 3, to develop this project with the aim of it being extended.

The *BitReader* class is responsible for decoding binary data streams into usable values. Internally it maintains a bit-level cursor (*bitPos_*) which tracks the current read position within the input buffer. Data extraction is performed one bit at a time using a masking function *bitMaskInBytes* ensuring correctness regardless of bit ordering. Higher-level helper methods such as *readU16*, *readU32*, and *readU64* reconstruct multi-byte values according to the configured byte order, while *readVarUInt* implements a variable-length integer encoding scheme similar to LEB128 (Chen, 2024), enabling efficient transmission of small values. Error handling is explicitly managed through an internal error state, preventing undefined behaviour in cases such as buffer overruns or misaligned access.

Conversely, the *BitWriter* class provides the inverse functionality, allowing structured data to be encoded into a compact binary representation. Like the *BitReader* class, it tracks a bit-level write position and dynamically resizes the underlying buffer to accommodate new data. The *writeBits* function performs fine-grained insertion of values into the buffer, writing from the most significant bit of the field to the least significant bit to ensure consistent reconstruction. The writer also provides alignment utilities (*alignToByte*) and supports both fixed-width and variable-length encodings, mirroring the capabilities of the reader. This symmetry ensures that any data written by the system can be reliably parsed without ambiguity.

Supporting these core components are several utility classes. The *ByteBuffer* class wraps a *std::vector<uint8_t>* to provide controlled access and resizing semantics, while the *Span* class offers a minimal, non-owning view over contiguous memory, avoiding unnecessary copies during parsing. Additionally, an *Error* enumeration class standardises failure states across the library, improving robustness and debuggability.

3.1.3.3 Config

Contribution

This library loads and validates the configuration JSON file which is copied into each of the respective component Docker containers during deployment, and safely expose the configuration values to the C++ binaries.

Implementation

The library is built using the open source *nlohmann* json library, discussed in 3.1.5.3.

The *Config* class parses the configuration file variables and information into structs, stored under the *shared::config* namespace. These structs (such as *Engine* and *Transmission*) store the hardware profiles/ attributes and organise the data into structures the rest of the code are able to use.

The *Config* library validates the data passed into the config JSON, checking things such as gear size etc., throwing runtime exceptions if these are not correct.

```
const Engine& getEngineConfig() const;  
const Transmission& getTransmissionConfig() const;
```

Figure 7 Safe accessor methods to access the configuration values

The library also provides safe accessors as shown above in Figure 7, which expose the configuration values in other parts of the code. These accessors return const references, which prevent accidental modification and enforce read-only access after loading the data. By returning references, I am also preventing potential expensive copies of the *Engine* and *Transmission* objects. Furthermore, by making the functions themselves const, it is enforcing that none of the member attributes in the class i.e. the configuration values are changed, further illustrating the intention of these functions and how they should be used when using them to develop code.

By developing this Config library, accessing the config.json file is much safer and easier to maintain throughout the code, creating a single point of maintenance and configuration provider. This library means that it is easier for me to update the C++ code when changing the configuration JSON, and the logistics of this parsing logic are abstracted from the main code. This library also allows for the ability to test the configuration parsing code and logic in isolation, which formed part of the unit test framework.

3.1.3.4 Logging

Contribution

To aid the debugging and runtime traceability of the NEON system, a dedicated logging library was developed which provides each container with a consistent, thread-safe mechanism for recording and outputting diagnostic information during execution.

Implementation

The *LogFile* class is implemented as a singleton, accessed through a static *instance()* method. This design ensures that each container process maintains a single, globally accessible logging instance throughout its lifetime, preventing duplicate log handles or conflicting file writes across threads. The singleton is initialised lazily on first access, meaning no logging overhead is incurred unless the system actually calls it.

The library is developed to support four log levels: *Debug*, *Info*, *Warning* and *Error*. These are stored in a *LogLevel* enum. A configurable minimum level threshold can be declared at runtime, an example being that only Errors can be logged. This helps keep output focused and meaningful during development and would be used for demonstrations.

Each log entry is prefixed with a millisecond precision timestamp, generated using *std::chrono::system_clock*. This level of granularity is important in real-time, multi-threaded simulations where events across the different components, such as publish data or receiving CAN messages, may happen almost simultaneously. This was especially useful for debugging and traceability of the NEON solution. An example log line output is shown in Figure 8 below.

```
2025-04-01 14:32:07.413 [INFO] ECM: CurrentGear message received
```

Figure 8 An example log line output from a NEON container

Thread safety throughout the class is enforced using a *std::mutex*, locked via *std::scoped_lock* within the *log()* function and *std::lock_guard* within *setLogFile()*. This ensures that concurrent log calls from multiple threads within the same container do not interleave or corrupt the output stream.

Finally, the library exposes a set of concise static methods, such as *LogFile::info()* and *LogFile::debug()* etc, which delegate directly to the singleton instance. This keeps code clean and readable and removes the need to retrieve the instance explicitly at every usage point, reinforcing the goal of being a lightweight and easy to integrate library.

3.1.4 Software Development Tools

This section of the report will cover some of the surrounding software development tools which were used and implemented throughout the development of NEON.

3.1.4.1 Git & Version Control

Git and version control has been essential for managing the development of SAS. It provides a structured way to track changes over time, allowing for the experimentation of features whilst new code is confined within the isolation of stand-alone branches. This adoption of version control helps ensure that the main branch, and final solution, is kept well tested and robust.

Version control also acts as a safety net, enabling actions such as revert, bringing the code back to previous working states if issues arise, which is particularly valuable when working across a mixed environment involving Linux, Docker containers, and Visual Studio toolchains. Finally, Git serves as a form of documentation, giving a clear history of design decisions and implementation progress, which is helpful for debugging, reporting, and future extension of the project.

Git was chosen for this project as it is widely used and easy to implement within different operating systems and software applications such as Visual Studio.

3.1.4.2 CMake

CMake is a cross-platform build system generator that produces native build files, such as Makefiles or Ninja build scripts. These build files define the rules for compiling source code, managing file dependencies, and linking programs. By abstracting this process, CMake simplifies the creation and maintenance of build configurations.

CMake was chosen as the primary build system to ensure portability, scalability, and long term maintainability across platforms. As this project was developed on both Windows and deployed to Linux, CMake enables the generation of platform-specific build files using appropriate tools (e.g. Ninja on Linux) from a single configuration. This eliminates the need for separate build scripts for each platform, significantly simplifying development and testing.

3.1.4.3 Operating System and Program Environment

For the development of the C++ code Microsoft Visual Studio 2022 was used, this application provides extensive support including a wide variety of extensions to accommodate the varied range of languages used within NEON, and is widely used within the software development industry.

The VSP is deployed as a Windows .NET application, meaning the C# is deployed using Windows operating system and the Roslyn compiler (Microsoft, n.d.), a modern C# compiler which is a part of the .NET SDK.

NEON was developed on both Windows and Linux. The code was deployed on a Linux Ubuntu system whilst using Windows Subsystem for Linux (WSL) to facilitate the Linux environment on a Windows desktop. This introduced several challenges, such as handling cross-platform differences in file paths and line endings, whilst also resolving discrepancies between the Windows MSVC toolchain and GCC/ Clang on Linux, and managing CMake configuration differences. Docker networking also behaves slightly differently, for things such as hostname resolution, under WSL compared to native Linux, requiring additional testing.

However, this approach also provided significant benefits, it allowed for a consistent Linux deployment target while still using a Windows based development workflow and IDE, making it easier to edit and develop code during development. WSL offered near-native Linux performance without requiring a separate machine or use of virtual machines, which if the scope of the project had allowed would have been beneficial to implement. Another benefit is that it ensured the NEON system remained portable for real Linux environments.

3.1.4.4 Docker & Docker Desktop

Docker is a tool used to provide development environments and containerise code into small, lightweight containers in which the code can run in.

Docker was used in conjunction with Docker Desktop to containerise each component and ensure a consistent, reproducible runtime environment across development and deployment. Docker allowed for the packaging of C++ binaries, dependencies, networking configuration and runtime settings into isolated containers. This eliminated any machine specific issues and helped illustrate the goal of NEON, that each component is independent

and can be interchanged with interoperable hardware to provide an effective testbed for testing new hardware.

Another approach would have been to employ virtual machines. Compared to full virtual machines, Docker is significantly lighter weight making it faster to start and much more resource efficient. This aspect of Docker made testing NEON especially easier as I was able to quickly deploy each component within a Docker container, providing identical environments each iteration for the C++ binaries to run.

Docker Desktop (Docker, n.d.) is the host of Docker on Windows. Docker Desktop provided an integrated environment that manages the Docker engine, networking, image storage and the WSL integration automatically, this could also be used as an alternative from the command line for changing and editing the containers which was helpful when testing the NEON system.

3.1.4.5 Programming Languages

C++

C++17 was selected as the primary language for developing the NEON simulations, striking a balance between the modern language features required for a system of this complexity and the stability and compiler support expected of production-grade software.

C++ was chosen over alternative languages such as Python for its deterministic timing which was crucial in car simulation. Whereas Python is predominantly used for scripting and data analysis, C++ is more commonly used in industry for embedded systems, so it made sense to use it on this project.

Furthermore, C++ is a well documented language and allows the programmer a lot of control when writing code, ensuring maintainable, clear and explicit code can be developed. Languages such as Rust were also considered, however, I opted for C++ over Rust because of its maturer ecosystem and the greater volume of documentation revolving around the language which is available.

Bash

Bash was chosen to write shell scripts as it is a very popular, industry used language which is well documented, and I have had industry experience using it within safety critical solutions in the past.

As part of this project, a variety of Bash scripts were created which were both incorporated into the project and also to aid the software development process. To deploy a vehicle a *deploy.sh* script is called which builds the code, configures the Docker network, creates the containers and copies the correct files into the containers.

I also created multiple Bash scripts to aid development and run frequent scripts. Scripts such as *enter.sh* and *clean_docker.sh*, were used to perform menial terminal tasks, massively speeding up development and automating monotonous tasks.

3.1.5 External Libraries

This section of the report will cover some of the external libraries that were incorporated into the NEON C++ code, their uses and why they were chosen to be integrated into this project.

3.1.5.1 *Boost/ Boost Asio*

Boost (boost, n.d.) is a widely respected collection of peer-reviewed, portable C++ libraries, with Boost.Asio specifically serving as a robust, cross-platform framework for network and low-level I/O programming. Designed around a consistent asynchronous model, it is built to support high-performance, concurrent C++ systems, making it an ideal choice for my NEON car simulation system.

NEON used two components from the Boost library suite: Boost.Asio for low-level asynchronous I/O and networking, and Boost.Beast, which builds on top of Asio to provide HTTP and WebSocket support. Together, these allowed the system to handle concurrent network operations without ever blocking the main simulation loop, adhering to NEON'S software requirements for maintaining consistent real-time behaviour.

Boost.Beast was used to bridge NEON and the VSP, managing the TCP connections that underpin the local HTTP server and upgrading specific endpoints, *carData* and *carCommands*, into persistent WebSocket streams. Through these, the simulation continuously pushes live JSON telemetry to the VSP every 50 milliseconds while simultaneously receiving control commands from the user interface.

The primary advantage of using Boost was the ability to handle UDP, TCP, HTTP, and WebSockets within a single, consistent framework, significantly reducing the amount of networking code required. The main trade-off was compile time, including the large Boost headers noticeably increased build times, though this was an acceptable cost given the functionality gained.

3.1.5.2 *googletest*

The googletest (Google, n.d.) library is a widely adopted, well maintained unit testing framework for modern C++, originally developed internally at Google to support testing of large-scale, performance critical C++ systems, making it relevant to my project particularly with the idea of performance being ingrained into my project and the code being developed. This library allowed for the quick creation of effective and dynamic tests for my C++ library code easily using built-in functions which are easy to add to.

Googletest was chosen as I was able to get a lot of experience with it during my industrial placement year, working with safety critical, real-time software solutions at Leonardo. This experience meant that I was familiar with the library, its capabilities and how to get the most out of it. These tests were imperative when dealing with shared library code, used in multiple different instances and proved helpful when debugging any issues.

3.1.5.3 *nlohmann*, JSON Parser

Nlohmann (MIT, n.d.) is a widely adopted, open-source C++ library that provides full JSON parsing, serialisation, and manipulation functionality. It is a header-only library, meaning no separate compilation or linking step is required, the library is simply included as a header file within the relevant translation units. This significantly reduces CMake configuration complexity, as no additional library targets need to be defined or linked across the different NEON components.

Within NEON, this library is used by the Config library (section 3.1.3.3) to parse the configuration JSON file, see an example at *appendix D.1*, that is generated by the VSP and copied into each Docker container at deployment time. The nlohmann library allows the structured JSON data to be read and mapped directly into the *shared::config* structs, providing a clean and readable interface for accessing configuration values throughout the codebase.

The decision to use nlohmann over alternative JSON libraries was made on several grounds. Compared to RapidJSON, another commonly used C++ JSON library, nlohmann provides a significantly more intuitive API that integrates naturally with standard C++ types and idioms, reducing the amount of boilerplate required to extract and validate values. RapidJSON prioritises raw parsing performance, however, given that the configuration file is parsed only once at startup rather than continuously, throughput was not a meaningful concern for this use case, making nlohmann's cleaner interface the more appropriate choice.

The option of writing a bespoke JSON parser was also considered and rejected. The configuration file contains nested objects and typed fields, all of which would require substantial implementation effort to handle correctly and safely. Developing and maintaining a custom parser would have introduced a disproportionate amount of code and testing overhead for what is a peripheral concern within the wider system. By contrast, nlohmann has been widely adopted across the C++ community, with extensive testing and a well-maintained issue tracker, meaning its correctness and robustness can be reasonably relied upon.

3.1.6 Coding conventions

In this section of the report, I will walk through certain coding conventions I have used throughout the NEON system, why I have made these conscious decisions, and what the reasoning behind them is. These are following my own industry experience with coding standards such as MISRA and DO178-C and relevant publications, such as C++ Coding Standards (Alexandrescu, 2005), which covers a range of topics at length.

3.1.6.1 Include Guards

Throughout my C++ within NEON, I used include guards in the format shown below in Figure 9. Include guards are a technique which prevent multiple inclusions of the same header file in a single translation unit, avoiding redefinition errors.

```
#ifndef SHARED_CONFIG_ENGINE_H
#define SHARED_CONFIG_ENGINE_H

namespace shared::config
{
    class Engine
    {
        // Class declaration
    }
}

#endif
```

Figure 9 Example of the relationship between include guards and namespaces

I chose to use this format of include guards over *#pragma once* for multiple reasons. The first is that *pragma once* is not included in the C++ standard preprocessing model, this would make my code less portable and compiler dependent. *Ifndef* guards are universally compiler supported, and handle path edge cases much better, they are also more traditional within C++ projects.

3.1.6.2 Decorators

Throughout the code, I utilise C++ language decorators such as *[[nodiscard]]*, these are vital in large scale coding projects, and show clear intent for functions and give a framework for how they should be used. This helps contribute to keeping good programming practices and tells the compiler when to throw relevant warnings.

3.1.6.3 Comments

Using my experience within the software industry, and from what has been taught to me during professional C++ courses, I have made the decision to keep comments to a minimum throughout my code. This is following the philosophy that good code should be self documenting and written using the Object Orientated Programming principles following key techniques such as polymorphism, inheritance, abstraction and encapsulation such that the code is easy to follow and understand. This process has been documented in publications (Martin, 2008) and adopted throughout the industry. I agree with this and think that plain English comments can often get messy and mis-interpreted as opposed to code and correct documentation.

3.1.6.4 File System and Namespaces

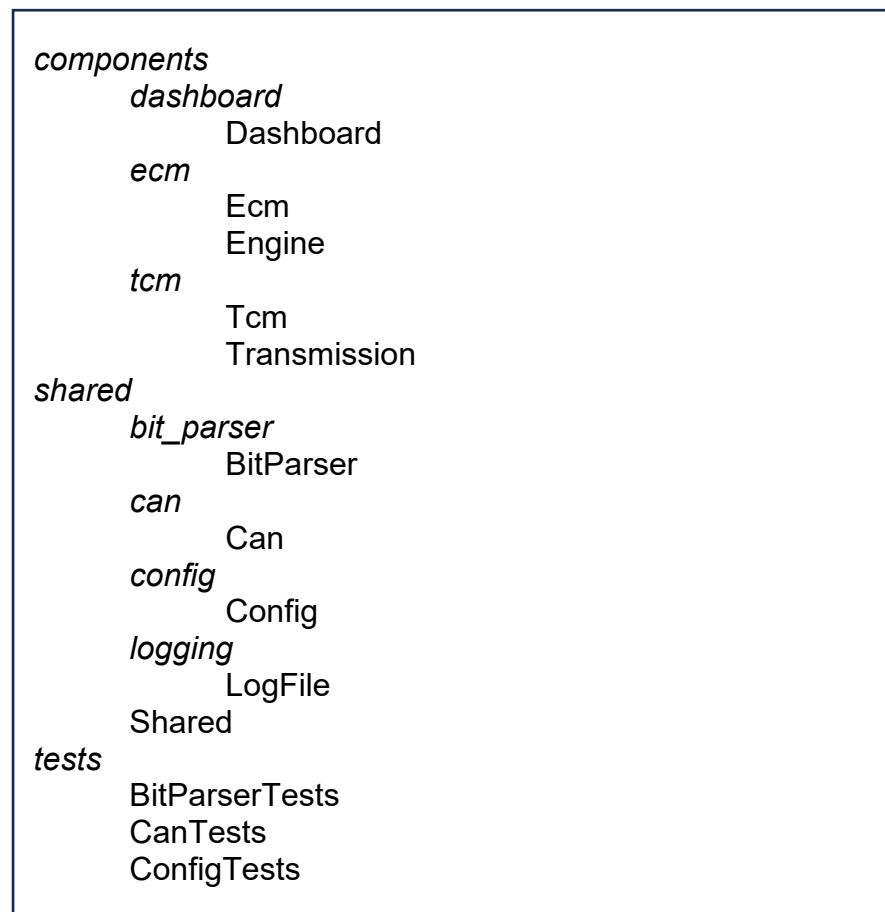


Figure 10 CMake file structure for the NEON project

As shown in Figure 10, where files are shown in *italics*, and CMake targets are shown in capitals, I have organised my code into a clear and maintainable file structure. CMake targets consist of libraries and executables. This structure helped keep my code organised, making development and linking files within my code easier and whilst ensuring scalability.

3.1.6.5 Class Member Conventions

```
#include <stdint>

class Person
{
public:
    [[nodiscard]]
    std::uint16_t getAge() const;
    std::uint16_t m_age;
private:
    std::uint32_t m_phoneNumber;
}
```

Figure 11 Example class, highlighting the naming conventions of member functions and attributes

As shown in the example above, I use lower camelcase for the function names, and prefix any member attributes, regardless of their protection level with a 'm_' this is to indicate to the developer what is a member attribute. To keep to traditional coding standards I do not use global variables, as these pose scoping issues and are often prohibited in safety critical systems and coding standards, including the DO178-C standard (Parasoft, 2011), which I have experience with from my year in industry. I have implemented a lot of these coding standard conventions into my code throughout development, this is to keep this project with a industry and embedded system focus.

3.2 Vehicle Simulation Platform

In this section of the report I will discuss the implementation of the Vehicle Simulation Platform (VSP). The main purpose of the VSP is to interact, control and monitor the NEON system.

3.2.1 Graphical User Interface Designs

The VSP was designed to be a clear, easy-to-use professional application, with a minimalist but functional set of displays and controls. This section will walk through the design decisions of each page within the VSP.

3.2.1.1 Deploy Page

Upon opening the application, the Deploy Page is the first page the user is navigated to. Before deploying the NEON system, this is the only page the user can see and interact with, preventing any confusion for the user.

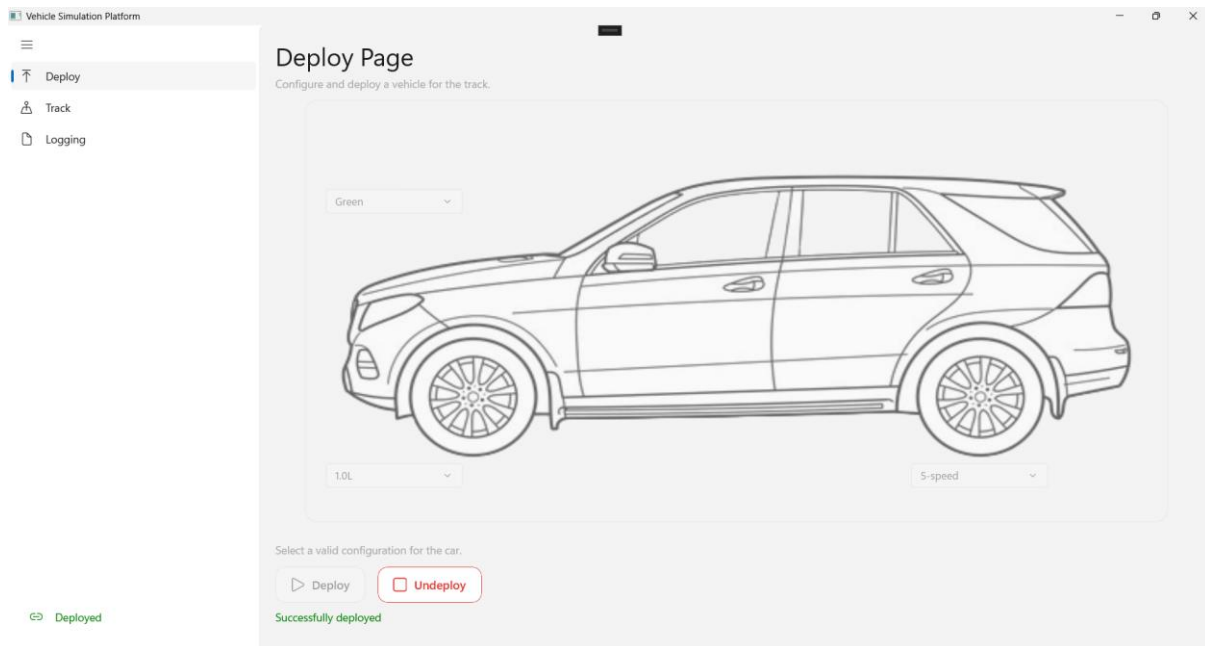


Figure 12 The VSP Deploy Page after successfully deploying the NEON system

This page, as shown in Figure 12, consists of a wireframe drawing of a car, to clearly show the intent of the page and link it with the automotive nature of the NEON system. Around the car diagram are the configurable aspects of the car, including the colour, engine type, and transmission system. These are drop-down menus so the user can clearly see what options are available and can only choose valid inputs. Only after each configurable is chosen will the deploy button display bright green and enable, indicating to the user that the NEON system can be deployed. Upon pressing the deploy button, a message comes up to indicate that NEON is "deploying". When the deployment is finished a suitable text appears, as shown in Figure 12, the deploy button disables, and cannot be reenabled unless undeployed and reconfigured, and in the bottom corner the status of the system updates from Offline to Deployed. Finally the other two pages (Track and Logging) both become visible.

3.2.1.2 Track Page

The track page is the crucial part of the VSP, this is where the user sends commands directly to the NEON system, and observes real time updates via the dashboard. It is shown below in Figure 13, and includes a graphic of a car on a road. This car "moves", through the use of the road markings, depending on the vehicle speed to give the user an additional visual aid to gauge their vehicle speed. It also features a control panel and the dashboard display on the far right. All the data is updated continuously to give a dynamic and useful display.

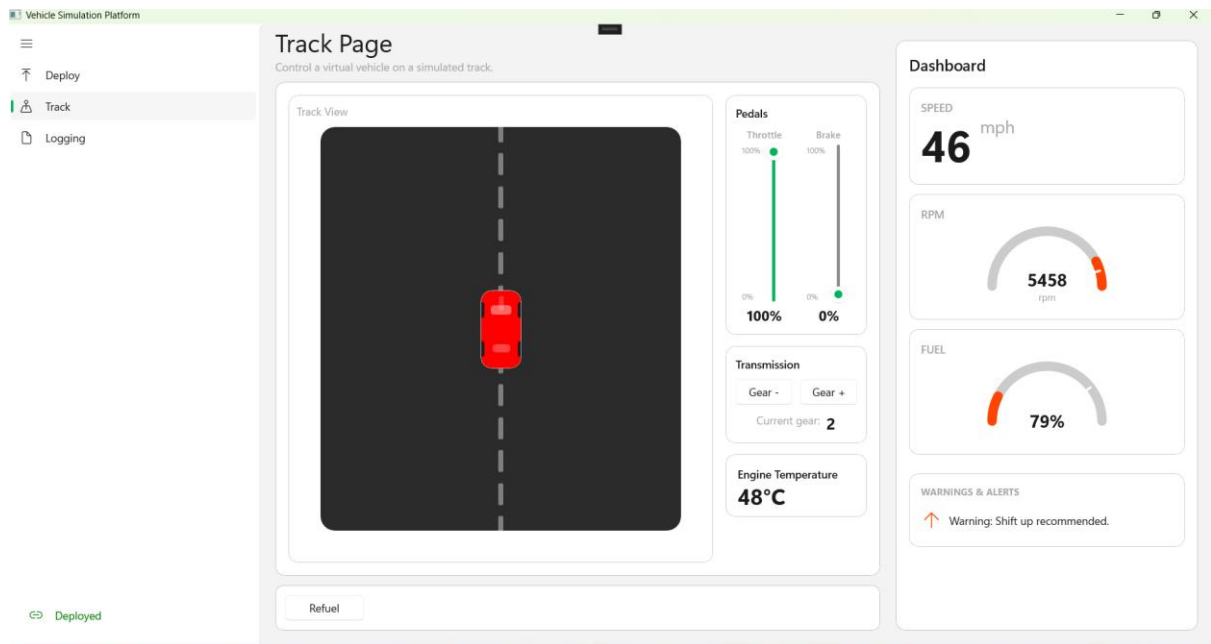


Figure 13 The Track Page on the VSP

3.2.1.3 Logging Page

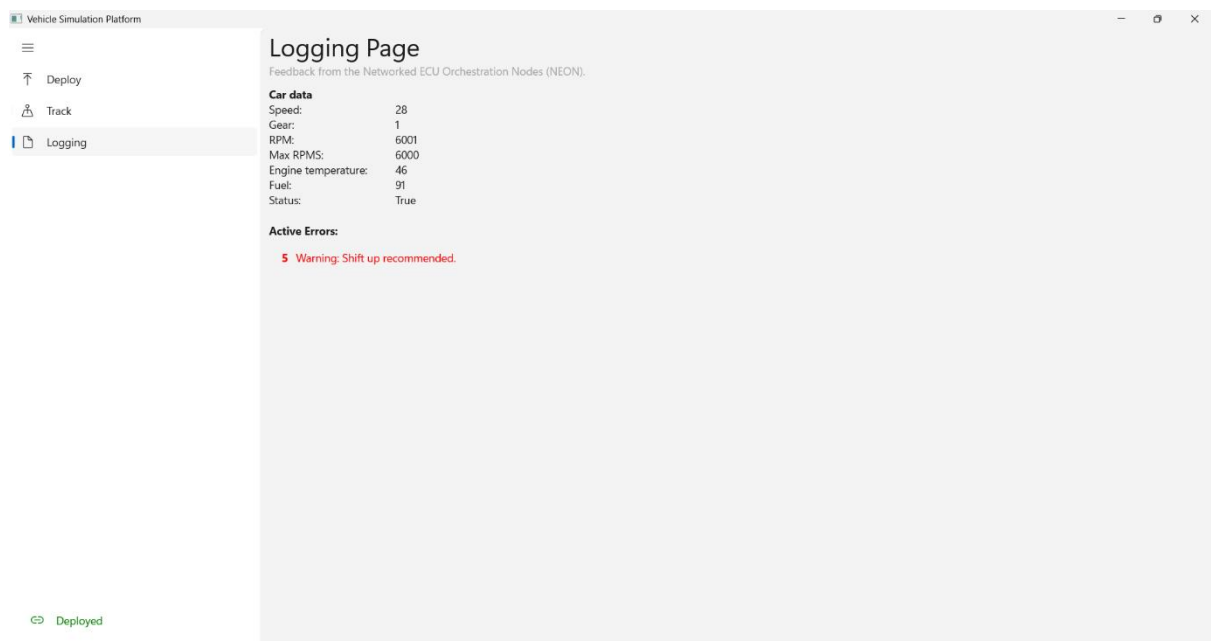


Figure 14 Screenshot of the logging page

The logging page, as shown above in Figure 14, is a page designed for the developer of the program to see what exactly is being sent from the NEON system, before it is parsed and changed for a graphical user display.

3.2.1.4 NEON Telemetry Window

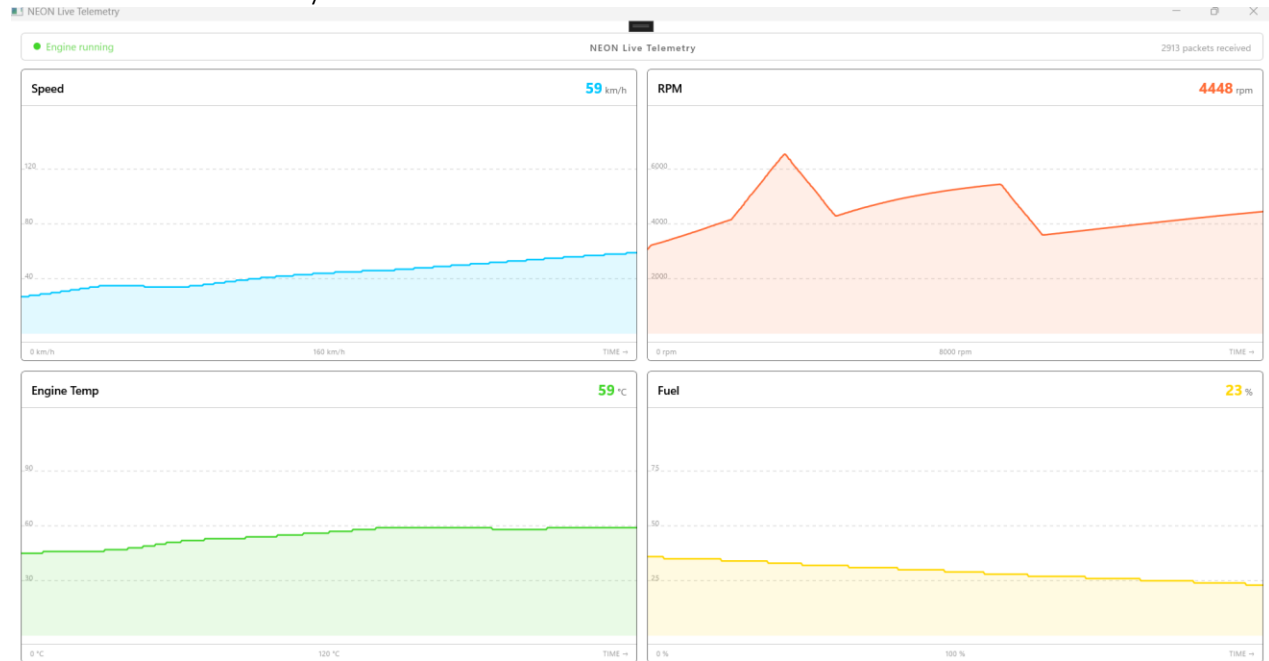


Figure 15 Screenshot of the NEON Live Telemetry Window

The telemetry window, as shown in Figure 15, is a flyout window option which can be opened and accessed from the Track Page. On this page it includes 4 graphs displaying the RPM, Speed, Fuel percentage and engine temperature, updating in real-time to display these attributes graphically to illustrate how they change as NEON is interacted with. This was developed to help validate the behaviours of my simulations and highlight any anomalous behaviours. It also displays the number of packets received from the NEON system, verifying the NEON connection and showing the velocity of which the NEON data packets are received.

3.2.2 Architecture

For the VSP I followed the Model-View-Controller (MVC) architecture (Fowler, 2003), a structural design pattern that decouples an application into three main logical components: the model which handles the data, the view which is the face of the application and renders the data into a graphical state, and the controllers which sit between the model and the view and dictate the actions and updates between them.

I chose to use this architecture because it lends itself to scalable applications where updates can be made in isolation and can easily be tested. I was also able to get industrial experience using this architecture during my industrial placement, where I learnt first hand how important this separation of logic and data is when maintaining a solution.

An alternative architecture would have been to use the Model-View-View-Model (MVVM) architecture, this is similar to MVC but adds a view model for data bindings, this is a slightly more complex architecture, which I didn't feel was needed for my application as the main role of the VSP is to simply control and monitor the NEON system. It would've also included more boilerplate code as each page would have required an individual view model, which is unnecessary and time consuming for a project of this size.

3.2.2.1 Model

The role of the model within MVC is to store the data which populates the View, and enable it for being accessed in a thread-safe format.

Within my solution I have four model classes:

- **CarConfig:** Used to store the car configuration details that are not sent to the NEON system such as the colour of the car, these are purely aesthetic and do not affect the simulation.
- **CarData:** This is the class that stores the information received from the NEON system about the car which is used to populate the View. This includes speed, gear, RPMs and error lists etc. This class uses locks to ensure thread-safety and handles any logic when the car data is being updated by the NEON system.
- **DashboardMessage:** This is the raw JSON data retrieved from the WebSocket to the NEON system. This is parsed and converted into *CarData*.
- **ErrorMessage:** This is a lightweight class to store error messages which consist of a numerical code and a string.

3.2.2.2 View

As previously mentioned my user interface consists of 4 pages, and these predominantly makeup the View. The role of the View is to just be graphical and display the data in the intended way. The View consists of XAML and C# code. These designs are discussed in 3.2.1.

3.2.2.3 Controller

The role of the controllers is to orchestrate the system and communicate between the View and the Model. For my solution I use 3 Controllers:

- **DeploymentController:** The DeploymentController mainly interfaces with the Bash scripts I have written to deploy and undeploy the NEON system. This class features functions such as RunWsIScript, which runs a script on the Linux command line, often calling a shell script and handles the return codes accordingly. Another role of the DeploymentController is to create the Configuration file during deployment, this file is first created in a local directory then copied into the working directory before finally being copied into the Docker containers.
- **VehicleController:** The VehicleController is a lightweight class to interface between the HttpClient (which is created from the VSP) and the NEON system, telling what the HTTP client should send to the system. These are sent as JSON values, which when received by the Dashboard binary are converted into CAN packets.
- **WebSocketController:** This controller connects to the WebSocket created by NEON, logs and parses any messages received from it in an asynchronous loop and handles a graceful disconnect when NEON is undeployed.

3.2.3 .NET Framework

I used the Microsoft .NET Framework to develop the VSP as a Windows desktop application. This is because I have previous experience using this framework and it is simple to implement and allows for a lot of features. It is also a widely used framework, with extensive documentation and supporting libraries, making it very versatile, and simple for implementing an application like the VSP. Furthermore, developing my code using Microsoft Visual Studio Code, the combination of these works very well, and MSVC has plenty of

additional functionality and tools such as the designer which make development simpler and quicker.

3.3 UML Interaction Diagrams

3.3.1 Configuration and Deployment

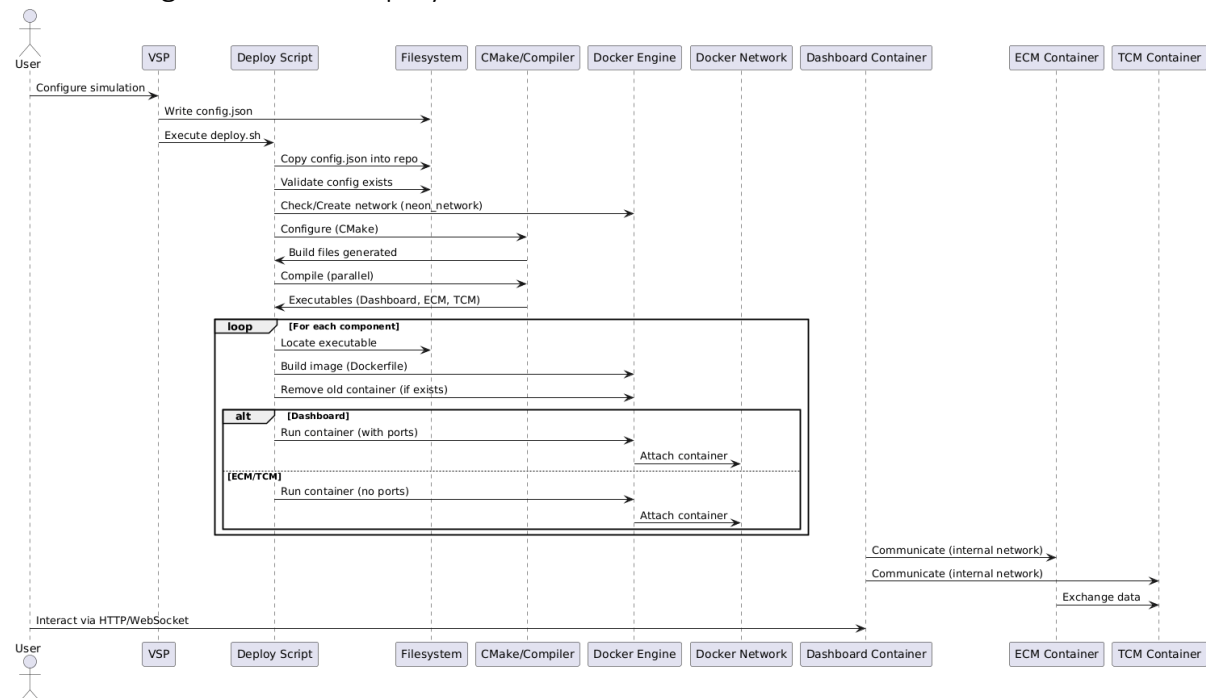


Figure 16 UML Sequence diagram illustrating the deployment process

Figure 16 shows the deployment process for NEON which is triggered via the VSP. When the user selects a desired vehicle configuration, the VSP generates a configuration JSON file. This configuration file contains an engine and transmission profile, derived from the configuration options, an example can be found in *appendix D.1*.

The deployment script is then executed, which first validates and copies this configuration file into the project workspace to ensure consistency across all the components. It proceeds to prepare the build environment by detecting the host operating system, creating a clean build directory, and invoking CMake to configure and compile the required executables for each component.

Once the binaries are successfully built, the script transitions into a containerisation phase using Docker. For each component, a corresponding Docker image is built using its dedicated Dockerfile, ensuring that all runtime dependencies are encapsulated. A virtual Docker network is created (if not already present) to simulate the communication backbone of the CAN networking protocol between each of the components. The script then launches each container, attaching them to the shared network and copying the shared configuration file as a read-only volume. The Dashboard container, which acts as an intermediary between the VSP and NEON, also exposes network ports to allow interaction and protocols such as WebSocket and HTTP between NEON and the VSP.

3.3.2 Gear Change Request Event

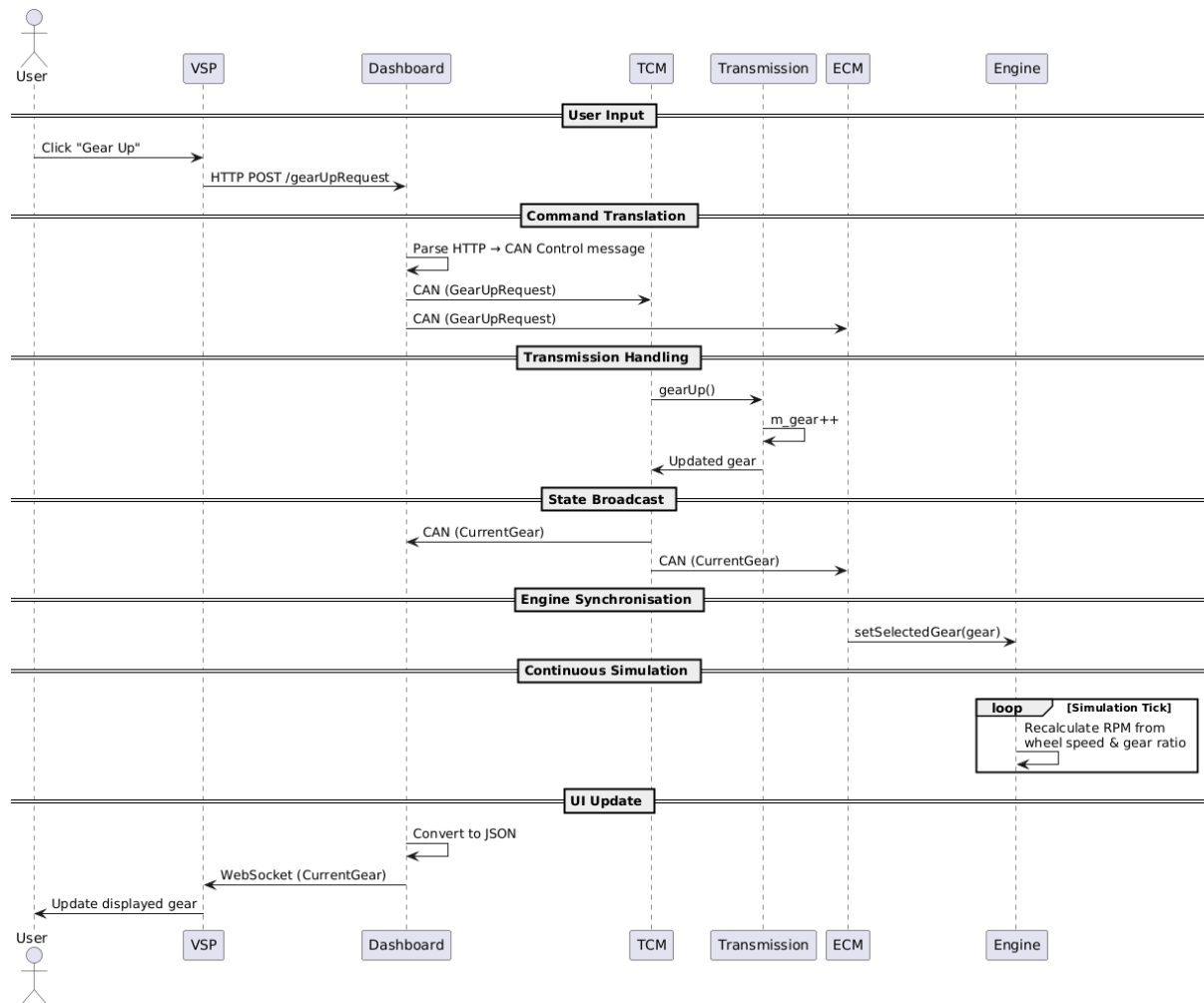


Figure 17 UML Sequence diagram displaying a gear change request

Figure 17 illustrates the runtime interaction between the core components of the NEON simulation during a gear change event, initiated via the VSP. The process begins when the VSP issues a gear change request to the Dashboard component. Upon receiving this request via a HTTP message the Dashboard parses this data into a CAN message, as defined within the ICD, *see appendix C.1*, the dashboard then broadcasts this across the network, with the correct message ID. As the transmission will subscribe to this message it will receive and log that it has received this message and update its local state of the current gear. The transmission also periodically broadcasts the current gear across the network. The engine subscribes to the current gear message and will as such, update its local copy of the current gear and this will then be implemented within the physics equations when calculating its data. This data is broadcasted around the network, the same as the current gear data, on either a fast or a slow telemetry loop, for speed, for example, it is posted within the fast loop at an interval of 10ms.

This continuous feedback loop ensures that the user interface remains responsive and accurately reflects the real-time state of the simulation, while also demonstrating the

decoupled, message-driven architecture of the NEON system, where each component operates independently but communicates through well-defined interfaces, which is outlined in the NEON ICD.

3.3.3 Car Stalling Event

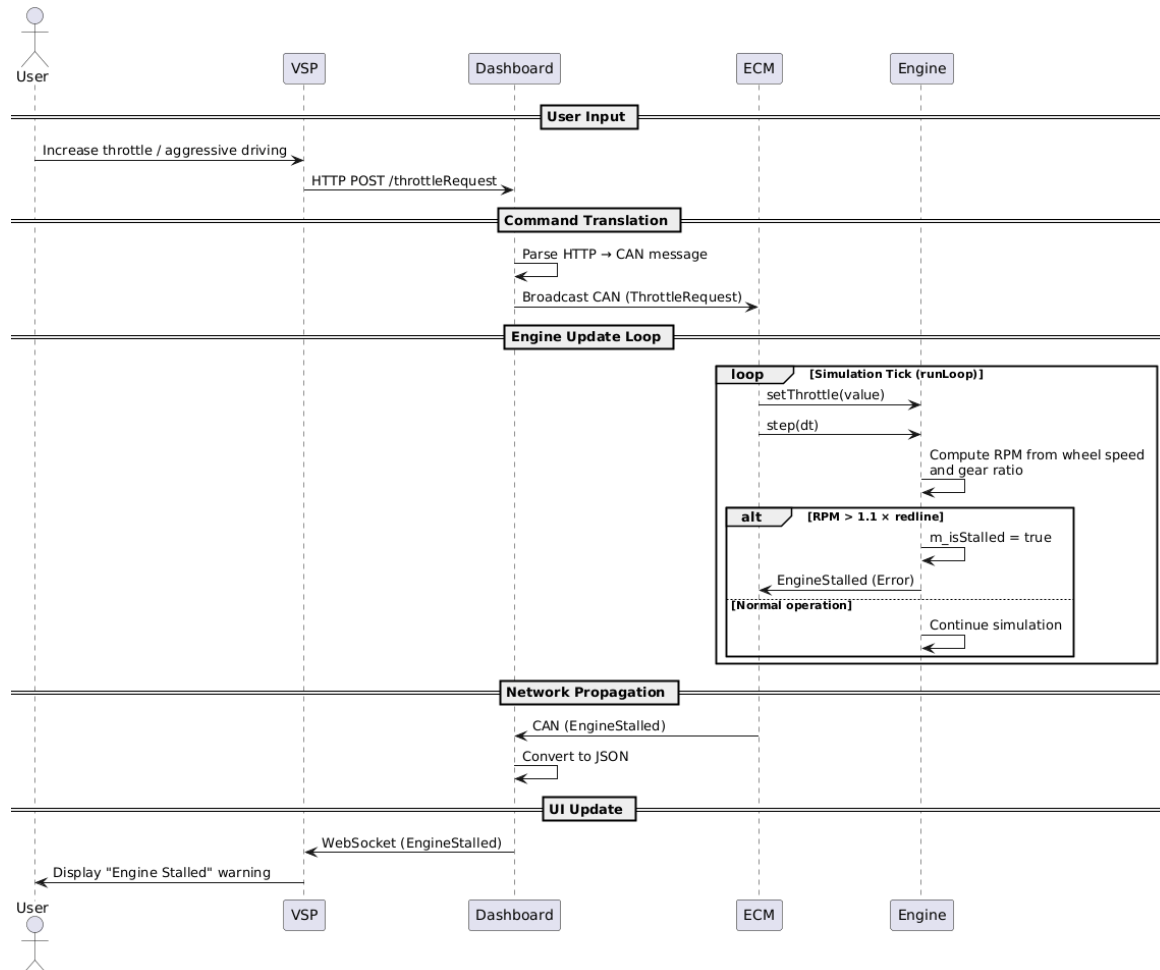


Figure 18 UML Sequence diagram showing the engine stalled event

Figure 18 illustrates the sequence of interactions that occur when the engine enters a stalled state due to mechanical over-rev within the NEON system.

The process begins with the user issuing an aggressive throttle input through the Vehicle Simulation Platform (VSP). This input is transmitted to the Dashboard component via a HTTP request, where it is parsed and converted into a CAN SpeedControl message. The Dashboard then broadcasts this message across the NEON network, allowing subscribed components, including the ECM, to receive and post the request.

Within the ECM, the Engine simulation operates continuously within a time-stepped loop. At each iteration, the engine updates its internal state based on current inputs, including throttle position, gear selection, and vehicle speed.

A stall condition is triggered when the calculated engine speed exceeds a defined mechanical threshold specifically 110% of the engine's maximum RPM. This condition

represents a catastrophic over-rev scenario, where the engine is assumed to suffer mechanical failure. Once this threshold has been hit, the engine sets an internal stalled flag, disabling further torque generation and effectively preventing the engine from responding to throttle input.

Following this state change, the ECM generates an error message indicating that the engine has stalled. This message is transmitted across the CAN network to the Dashboard, which converts the data into a JSON format suitable for the VSP. The Dashboard then forwards the update via a WebSocket connection, enabling real-time communication with the user interface. This error is then shown to the user on the VSP on the Track and Logging page, completing the feedback loop.

4 Testing

This section of the report will cover the testing of my project. This is crucial to verify and validate the correctness of my software, whilst contributing towards the evaluation and conclusive aims of this project. These tests have been developed, not only to prove the programs work, but also to find the bugs and errors within the code, following the Art of Software Testing publication (Sandler, 2012).

4.1 Unit Tests

To verify and validate the NEON code, I have written extensive unit tests, testing the libraries I have developed.

These unit tests were written using the *googletest* library, mentioned in 3.1.5.2, and are validated using a test coverage report and script in 4.1.1. By evaluating the test coverage using the *gcov* tool, this ensured these tests were written well and covered the variety of cases for each of my library code targets, which proved crucial in speeding up code development and project robustness.

I have not developed unit tests for any of the VSP code, as this is covered by the acceptance tests.

4.1.1 Test Coverage: NEON

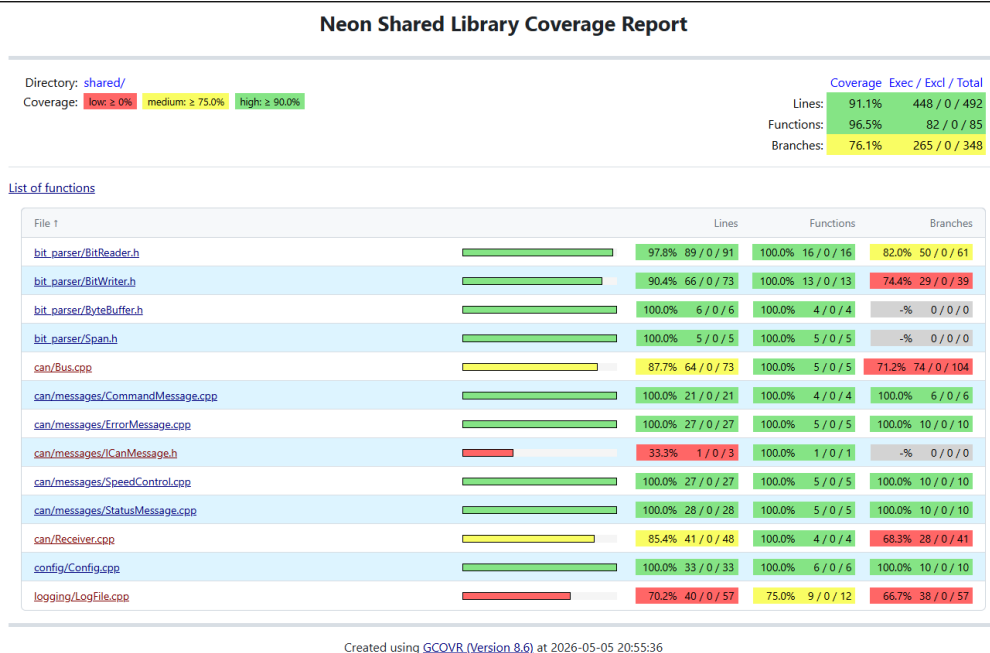


Figure 19 NEON unit test report generated by a Python script

In order to validate the test coverage for the NEON library code, a gcov tool and a Python script, *see appendix E.3*, was used to create a HTML unit test coverage report, as seen above in Figure 19. This was used during the development of the NEON code highlighting missed test cases whilst evaluating the effectiveness and depth of my unit tests.

I then went on to automate this process using a Python script, *see appendix E.3*. This script configured, built, tested and then collected the test data before converting it into a readable HTML report. These reports were checked and actioned upon at multiple stages of the development of the NEON code.

As seen from the report, I was able to achieve 91.1% of library code lines covered, and 96.5% of functions were covered. This is a very good result and shows the comprehensiveness of my unit tests. If I had more scope within this project I would integrate this Python script into a CI/CD pipeline to automatically run periodically, and aim to increase the test coverage to near 100%.

4.2 Acceptance Tests

In order to test the entire cohesive system, a range of acceptance tests has been developed. The aim of these is to deem whether the system performs acceptably and as expected, ensuring the robustness.

These tests cover a range of scenarios surrounding the software, and are top down centric, so focus on behaviours from the user interface and ensures that the entire system responds correctly. These tests have been derived from industrial acceptance tests I encountered during my industrial placement. In a professional project setting, with multiple contributors and periodic software deliveries, these tests would be ran periodically, and especially before

any deliveries. Throughout this project, these tests have been kept up to date, and ran before any big changes have been integrated into the master branch.

I developed clearly segmented and independent acceptance tests, so that different updates to the code did not require a complete run of the entire acceptance test suite, this practice helps speed up development and helps instill the knowledge of what parts of the code/ functionality are being changed with each update.

My acceptance tests were a great success and I was able to develop my software to pass all the acceptance tests. This illustrates my commitment to rigorous code testing, robust code development and my timekeeping skills within this project which were following the GANTT chart, see *appendix A.1*.

4.3 Static Analysis

To evaluate the quality, robustness and consistency of my code I have incorporated two static analysers throughout my development, configured to my own coding conventions, described in 3.1.6, which I have based on the coding standards I have encountered previously within the software industry such as MISRA , (MISRA, 2020), and DO178-C (Parasoft, 2011). These standards are in place to ensure that software conforms to certain guidelines and are often crucial in embedded systems to enforce good practices and reduce the risk of software failure through events such as memory leaks and stack overflow etc.

One of the static analysers I have used is Clang-Tidy, this is a compiler time analyser exclusively for my C++ code that increases the scrutiny of compiler warnings. This application integrates well with Visual Studio, allowing me to clearly see the warnings flag during development.

The other static analyser I use is ReSharper, (JetBrains, n.d.), I have experience using this and it is a highly regarded and powerful static analysis tool. This also runs within the IDE, so I can see warnings, errors and recommendations in real-time whilst writing code. I have configured this so it matches the coding conventions that I use and have found that this really aids me in ensuring my code is consistent throughout the project, and that I am making use of the powerful features of C++17 and C#.

4.4 Simulation Analysis

While unit tests verify the correctness of individual components and acceptance tests validate the overall system behaviour from a user perspective, the simulation analysis examines whether NEON produces physically plausible and consistent outputs under a range of driving scenarios.

4.4.1 Gear Up/ Gear Down

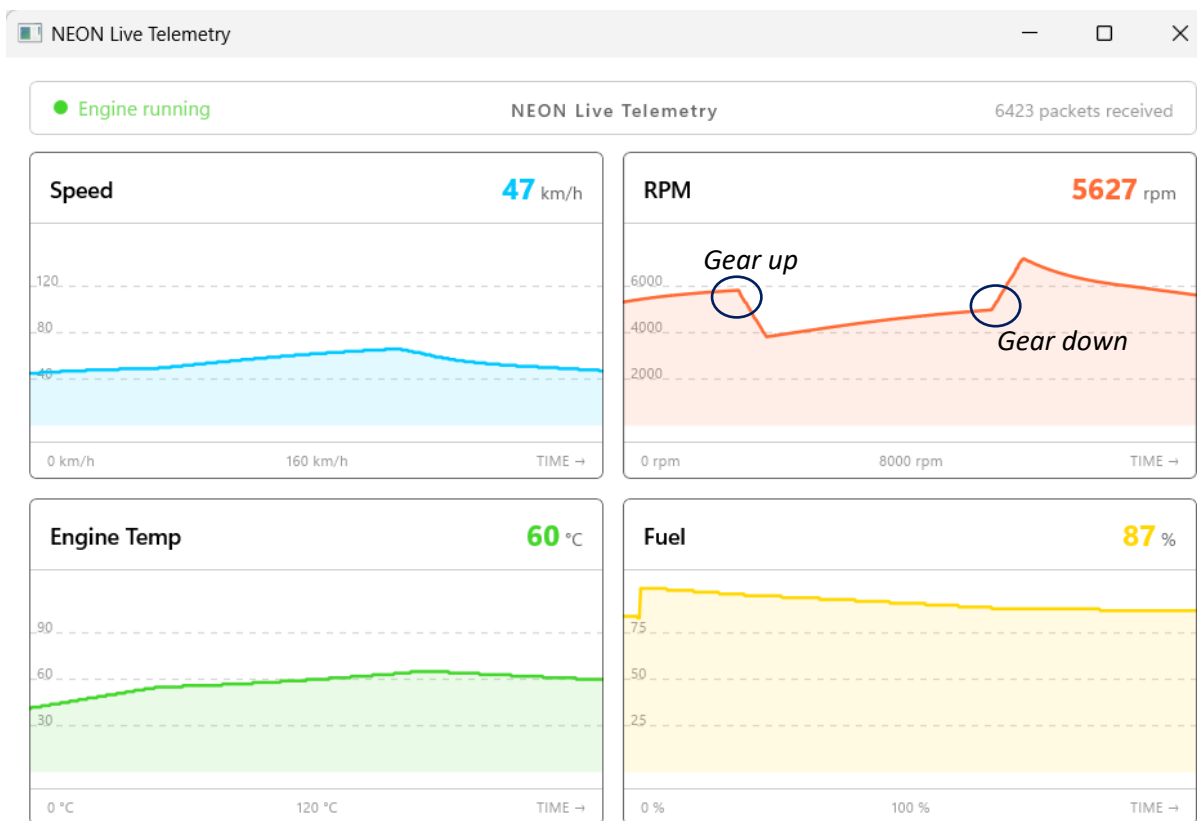


Figure 20 Snapshot of the NEON telemetry graphs responding to gear changes

When the vehicle is in gear and the throttle is held at a steady position, an upshift causes a measurable and immediate drop in RPM, shown after the *Gear up* label in the top right of Figure 20. This is consistent with expected behaviour: a higher gear ratio means the engine turns fewer times per wheel revolution, so the same road speed corresponds to a lower RPM. The speed remains stable immediately after the shift, as the drive force calculation recomputes using the new gear ratio, and the physics loop adjusts accordingly.

A downshift produces the inverse effect: RPM rises sharply as the same wheel speed now demands more engine revolutions, shown with the *Gear down* label in Figure 20. If the downshift is performed at high speed, the RPM spike observed in the telemetry window approaches the redline, which is physically plausible and consistent with the engine braking effect seen in real vehicles. If the RPM spike exceeds 110% of the redline, the stall condition described in section 3.1.2.1 is triggered, producing the *EngineStalled* error message on the network and disabling torque generation, this behaviour was verified by acceptance tests to function correctly and consistently.

The *ShiftUpRecommended* and *ShiftDownRecommended* messages were also observed to appear at appropriate points, broadcasting on the CAN network when the engine operates outside an efficient RPM band.

4.4.2 Engine Acceleration

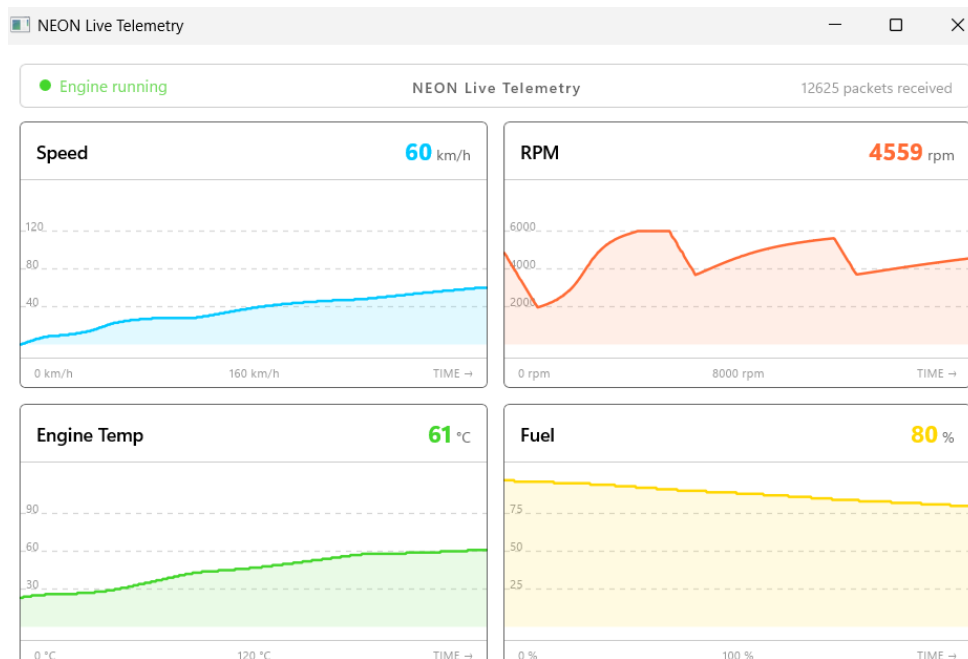


Figure 21 NEON telemetry graphs when accelerating from stationary

Starting from a stationary position in first gear with progressive throttle input, the simulation exhibits a characteristic acceleration curve, shown in the top left of Figure 21. Initially, as the engine climbs from idle towards its peak torque RPM (modelled at 60% of the usable rev range), acceleration increases. Beyond this peak, the Gaussian torque curve causes output to taper off as the engine approaches redline, reflecting the reduced volumetric efficiency of a real engine at high revs. This produces a natural, diminishing rate of acceleration at the top of each gear, which is the expected behaviour.

The fuel level was observed to decrease at a faster rate under heavy throttle, particularly as RPM normalisation approached 0.85, at which point the rich mixture multiplier of 1.15 applied by the fuel model caused a visible and slightly sharper reduction in the fuel percentage displayed on the telemetry window. This is consistent with the enrichment behaviour described in section 3.1.2.1.

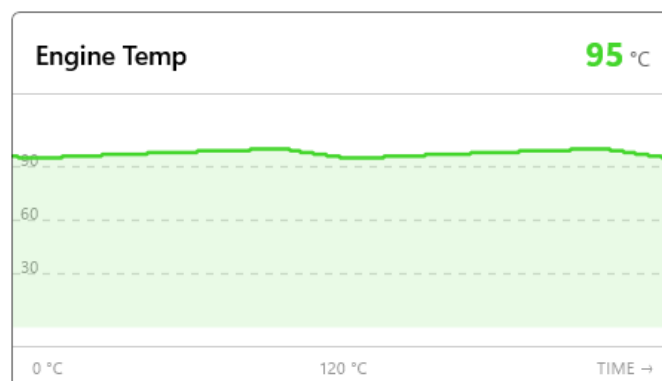


Figure 22 Snapshot showing the hysteresis model for engine temperature

Figure 22 demonstrates the *EngineOverheating* scenario and the hysteresis behaviour, described in 3.1.2.1. Engine temperature rises steadily under sustained acceleration, with the rate of rise increasing at higher throttle positions and RPMs, consistent with the combined friction and combustion heat model described in section 3.1.2.1. Under sustained high-load driving, the EngineOverheating error was observed in Figure 22 to trigger once the temperature exceeded the defined threshold, and crucially, the hysteresis implementation prevented the warning from flickering, it remained active until the temperature had fallen sufficiently below the threshold, confirming the robustness of the overheating detection logic.

4.4.3 Engine Braking

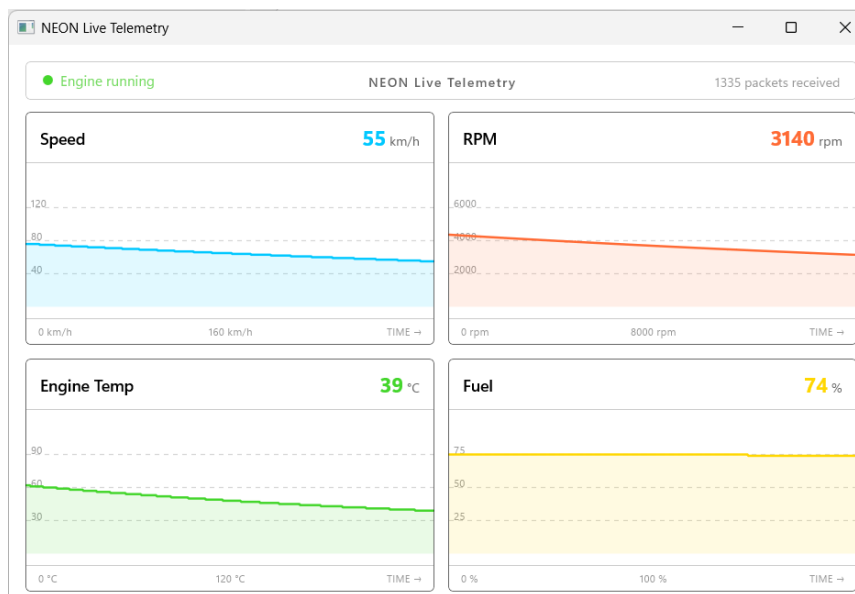


Figure 23 Telemetry graph displaying engine braking behaviour

Shown in Figure 23, with the vehicle at speed in a high gear, releasing the throttle and applying no brake input produces a gradual deceleration due to the rolling resistance and aerodynamic drag terms in the resistive force model. This is consistent with real-world coasting behaviour, where a vehicle slows progressively without the driver actively braking.

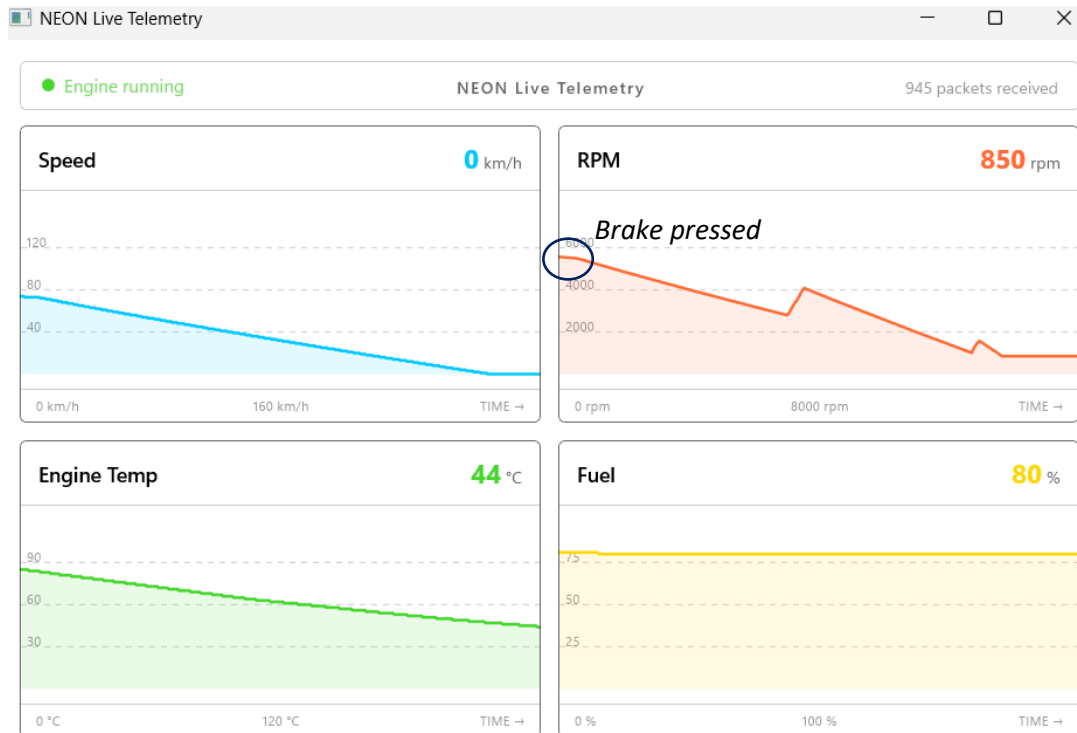


Figure 24 Telemetry graph displaying brake input behaviour

Figure 24 illustrates that when the brake input is increased via the VSP, a more rapid deceleration is observed, with the speed reducing to zero in a manner proportional to the braking force applied. RPM was observed to fall alongside speed when in gear, consistent with the gear-coupled RPM model where engine speed is derived from wheel speed rather than being independently controlled during braking.

Upon reaching a very low speed in a high gear under braking, the RPM dropped toward idle, and in cases where the speed continued to fall while in gear, the RPM approached the lower boundary. The simulation handles this gracefully, preventing negative RPM values.

5 Evaluation

This section critically evaluates the overall success of the SAS project in relation to the original requirements, as well as its technical strengths, limitations and relevance to real world systems.

5.1 Achievements of Requirements

This section evaluates the degree to which SAS has met the software requirements defined in section 2.2.

Requirement 1 — Graphical User Interface. The VSP fully satisfies this requirement and all of its sub-requirements. Requirement 1.1 is met by the deployment of the VSP as a Windows desktop application accessible via the Windows Start Menu. Requirement 1.2 is addressed through the design choices discussed in section 3.2.1, including the use of dropdown menus to constrain valid inputs, progressive page visibility that prevents the user

from accessing the Track or Logging pages before NEON is deployed, and a consistent colour scheme throughout the interface. Requirement 1.3 is satisfied by the Deploy Page, which allows the user to select vehicle configuration options and trigger the deployment of the NEON system via the `deploy.sh` script. Requirement 1.4 is met through the WebSocket connection maintained by the Dashboard, which pushes a JSON telemetry payload to the VSP every 50 milliseconds, populating the Track Page and live telemetry graphs in near real-time.

Requirement 2 — Automotive System Simulation. Both sub-requirements have been met. Requirement 2.1 is satisfied by the ECM, which simulates fuel consumption through the FuelTank model, power output through the Gaussian torque curve and drivetrain force calculations, and thermal behaviour through the ThermalModel class with its hysteresis-based overheating detection, all of which are described in detail in section 3.1.2.1. Requirement 2.2 is satisfied by the TCM, which manages gear state, responds to gear change requests received over the CAN network, and periodically broadcasts the current gear to peer modules as described in section 3.1.2.2.

Requirement 3 — System Architecture. The containerised architecture requirement has been fully met. Each ECU, the ECM, TCM, and Dashboard, runs as an independent C++ binary within its own Docker container, attached to a shared virtual Docker network. This design ensures modularity and directly supports the project aim of creating a test bed into which new components can be integrated or replaced in isolation.

Requirement 4 — Real-Time Operation. This requirement has been addressed through the use of time-stepped simulation loops governed by a steady clock, as described in section 3.1.2.1. By calculating a precise time at each iteration, the physics remains consistent regardless of network jitter or processing overhead. Asynchronous networking via Boost.Asio ensures that simulation execution is never blocked by I/O operations, which is an important consideration for maintaining responsive real-time behaviour.

Requirement 5 — Communication Protocol. Both sub-requirements have been met. Requirement 5.1 is satisfied by the CAN library described in section 3.1.3.1, which emulates the broadcast-style messaging of a physical CAN bus by transmitting serialised datagrams over UDP, removing hardware dependencies whilst preserving the structure and behaviour of the protocol. Requirement 5.2 is satisfied by the three message categories, Control, Status, and Error, and the full set of message types defined in the Interface Control Document (Appendix C.1), which together support the range of data formats exchanged across the NEON network.

Taken together, all five top-level requirements and their associated sub-requirements have been successfully implemented, demonstrating that SAS achieves the goals established in section 2.2 and the broader project aims outlined in section 2.1.

5.2 Technical Strengths

The architecture of this project is one of its key technical strengths. The decision to model ECUs as independent Docker containers introduces a high level of modularity and scalability. Each component can be developed, tested and replaced in isolation, closely mirroring

modern distributed systems used in industry. This design also aligns well with the concept of digital twins, where individual subsystems may be integrated with interoperable digital twins or real hardware.

Another major strength is the CAN simulation. By transmitting CAN over UDP, the system successfully emulates the broadcast based communication models which are employed by real-world automotive systems. The categorisation of messages into Control, Status, and Error types simplifies subscription logic and improves maintainability. Additionally, the use of a custom Interface Control Document (ICD) demonstrates an understanding of industry-standard documentation practices and highlights the attention to detail and commitment to scalability of the project.

The mathematical modelling of the engine is also a notable strength. The implementation of torque curves, resistive forces, fuel consumption, and thermal dynamics provides a physically grounded simulation. While simplified, these models are sufficiently detailed to produce realistic and responsive behaviour, enhancing the credibility of the simulation, which is then validated by my simulation analysis, covered in section 4.4.

5.3 Limitations and Challenges

Despite its strengths, the project has several limitations that impact its realism and scalability.

A significant limitation is the lack of validation against real-world data. The engine, fuel, and thermal dynamics are based on theoretical equations and approximations rather than empirical data from actual hardware. As a result of this, the simulation behaves plausibly, but it cannot be considered an accurate digital twin. This limitation was primarily due to the absence of access to real automotive datasets or hardware specifications. However, this project does still show the clear framework, and technical stack for hardware and the correct physics modelling to be implemented.

Another potential limitation lies in the networking model. While UDP was chosen to mimic the low-latency, connectionless nature of CAN, it does not fully replicate the key characteristics of a real CAN system, such as message arbitration, prioritisation, and deterministic timing guarantees. In real automotive networks, message collisions are resolved through hardware-level arbitration, which is not modelled in this system. Consequently, the simulation does not fully capture the behaviour of a true CAN bus under high load conditions.

From a user interface perspective, the VSP is functional and intuitive, however, this could be expanded with more features, displaying the data and creating reproducible and replayable “scenarios” to improve testability and the ability to verify and validate the NEON system. Despite this, it is still successful at controlling and monitoring the NEON system.

5.4 Comparison to Industry Systems

In comparison to real-world automotive simulation platforms, this project successfully captures several key principles, including modular ECU design, message-driven

communication, and the use of simulation as a testing tool. The use of containerisation and standardised interfaces reflects modern industry practices.

However, as I know from experience, digital twin and automotive systems typically operate under stricter constraints, including real-time, safety verified operating systems (some Linux distributions are not safety verified at certain safety levels) and safety critical standards. While this project does not fully replicate these constraints, it provides a strong conceptual and architectural foundation that could be extended towards more realistic implementations. I am also able to draw on my experience with aviation safety standards such as DO178-C (Parasoft, 2011), when developing my own code and writing the code conventions, making them more realistic.

6 Project Ethics

This project had an ethical category of A0, meaning that no human participants or data derived from humans or animals were used during the development of this project. As a result there are no ethical considerations to mention.

7 Conclusion & Future Work

Overall, this project was a success and it achieved in simulating an automotive, comprised of independently running and networked simulated components, which was able to be configured, controlled and monitored by an easy-to-use desktop application.

This project achieved the goals it set out to accomplish, by using and implementing industry standard languages, development tools, and software development processes, such as the CI/CD pipeline. Through experiences gained throughout my degree I was able to develop a full-stack piece of software that a large professional engineering company would be looking to implement and could expand upon. Throughout the development of this project I have stuck to a clear project plan, outlined in my proposal (see *appendix A.1*), built upon my industrial experience using C++ and working within different operating systems, adhered to consistent coding standards (described in 3.1.6), and used these technical skills to innovate and create my own networked interface of independently running components, comprising to form a full cohesive system.

On top of this, I was able to display and develop a further range of skills whilst achieving my predefined software requirements to create a intuitive graphical user interface, for which I utilised a .NET framework (Microsoft, n.d.) to create an aesthetic and easy-to-use desktop application. As part of this application, I followed Model-View-Controller architecture, highlighting my commitment to the core aim of this project which was to follow industry standard procedures whilst producing clear maintainable code in order to develop a scalable system.

Moving forwards with this project, with context to the nature of the software, integrating a piece of hardware into the project would be a great next stage in development. This will directly mirror the systems that large engineering companies are creating, by testing real hardware in virtual synthetic environments. Integrating real hardware would also verify my

own code, particularly the ICD I have developed (see *appendix C.1*) and ensuring that the technical stack and processes I have developed will be upheld when integrated with hardware.

If this project was to be deployed in a professional setting, as would be the intended use case of a system such as SAS. I would need to adhere to software safety standards, practices I have already begun to implement as described in section 3.1.6, drawing on past experience programming to a DO178-C (Parasoft, 2011) level. These safety standards use strict guidelines, such as not using options such as dynamic memory and external non-validated libraries within the codebase. This helps create much more robust, verifiable and validated code, which is paramount in real-time embedded systems, especially when they are deployed within integrated hardware test rigs.

Furthermore, if the project scope had allowed for it, I would have liked to have created CI/CD pipelines to run automated tests, using a server tool such as Azure DevOps or Jira to help evaluate my system, and speed up development. I have been able to accomplish some development speed up wins through the use of Bash and Python scripts to abstract certain functions and avoid menial tasks.

Overall, SAS creates a great initial test environment and framework for new automotive systems, employing industry standard techniques, languages and frameworks to create a reliable, and effective simulation platform. To conclude, SAS is a full-stack software solution boasting a clear and deliberate modular design, focused and purposeful bespoke libraries, precise supporting documentation, a robust and rigorous test framework, and an intuitive and professional GUI. This project has a lot of potential and clear vision moving forwards and has been a great success in regards to the aims and objectives outlined.

8 BCS Criteria & Self-Reflection

In this section of the report I will outline how I have met the six outcomes expected by the chartered institute for IT.

These are:

- An ability to apply practical and analytical skills gained during the degree programme.
- Innovation and/or creativity.
- Synthesis of information, ideas and practices to provide a quality solution together with an evaluation of that solution.
- That your project meets a real need in a wider context.
- An ability to self-manage a significant piece of work.
- Critical self-evaluation of the process.

An ability to apply practical and analytical skills gained during the degree programme.

As referenced throughout the report, the project was influenced by my experience during my industrial placement. I was able to see the need for digital twin software, which provides a far cheaper, more flexible and reusable test environment for new complex hardware solutions. This placement also gave me crucial software development experience which I have integrated into this project, developing critical documents such as Interface Control

Documents (ICDs) and acceptance tests. Without my placement I would not have the knowledge to include these and the experience to write them to a comprehensive standard.

Additionally, undertaking modules such as Computer Networking, and Cloud Computing for E-Commerce, I also became familiar with different networking protocols, the layers of the TCP/IP stack and the process of containerisation within software development, using software tools such as Docker and Docker desktop. This knowledge was critical when employing the range of networking protocols which support SAS.

Innovation and/or creativity.

Before this project, I had no experience with the inner workings of vehicles, or working with cars. I thought this was a great chance to learn more about the engineering which underpins automotives and their design.

As part of this project I had to create my own message definitions for how the components would communicate as defined in the NEON ICD, *see appendix C.1*. This ICD took a lot of revisions and changes as the project grew and more messages were sent around the network. I had to decide on serialisation techniques and standards, what data was actually being sent and how the data would be parsed around the network.

Finally, this project has also allowed me to create a desktop application, which is clear to a user and engaging, using colours and designs to influence intent and clarify user interaction. I developed a clear, and informative Track Page, which clearly shows what is happening to the NEON system, whilst formatting the telemetry data being sent across the network in a captivating way in real time. This Track Page can be seen in the earlier sections within section 3.2.1.2 of this document. The VSP also features a lot of dynamic moving features, such as graphs, a moving car animation, relating to vehicle speed, and dials which move in response to user input, all of these required knowledge of thread concurrency, the possibilities and limitations of XAML and C#, and bindings.

Synthesis of information, ideas and practices to provide a quality solution together with an evaluation of that solution.

Tools and Languages used:

- C++
- .NET (C#, XAML)
- Bash
- Python
- CMake
- Docker
- Git
- JSON

Networking Protocols:

- HTTP
- WebSockets
- CAN
- UDP/ TCP

Software Development Environments (SDE):

- Microsoft Visual Studio
- Microsoft Visual Studio Code (MVSC)
- Vim
- Vi
- Linux (Ubuntu)

SAS is a full stack complete software system which is the synthesis of a wide variety of languages, techniques and protocols, developed and deployed within a number of environments, as shown above. This has forced me as a developer, to be widely researched

in a range of applications and creative to bring all these ideas together. All these attributes of SAS have been derived and chosen from industrial experience and teachings from university, ensuring that I am developing my system in a true-to-form way of how a professional company would. The importance of a good foundational technical stack was highlighted to me throughout my placement consequently, I have chosen specific languages and SDEs for specific roles.

Furthermore, I have had to assemble these different aspects of the project, employing and designing a range of networking protocols and sequential behaviour patterns, described in 3.3. This took a lot of trial and error to get the near-real time behaviour seen within the final product.

Finally, I employed a range of testing techniques and code analysis tools in order to evaluate my system at each level of granularity, these are described in section 4.

That your project meets a real need in a wider context.

This project was aimed at creating a lightweight virtual test bed for automotive components to be tested against. My software allows for the integration of standalone independent systems, hardware or software, to be implemented and work off the architecture and code that has already been developed, following the digital twin philosophy, first introduced by Michael Greaves (Grives, 2016).

The need for this project is aligned with the rise of digital twin technology which larger engineering companies are quickly adopting with the digital twin market growing exponentially (Prime Nearshore, n.d.) . Digital twins are virtual representations of a physical object or system that uses real-time data to accurately reflect its real-world counterpart's behaviour, performance and condition (Gallagher, 2020), often modelling complex and expensive hardware, still in development, These systems allow hardware to be connected to a fully synthetic, bespoke system and environment, enabling complete and isolated testing. Furthermore, the development of these simulated environments can be recycled and reused in the future, encouraging future development and building off legacy/previous projects. I saw this first hand during my industrial placement, where the importance of testing and modelling complex and expensive hardware in simulated environments was crucial to the development and visualisation of these projects.

My software allows exactly this because it is developed in a clear, modular and containerised way, with supporting documents such as an ICD to aid future sub-system integration. SAS is completely customisable as a piece of software, and as mentioned, the development choices I have made aid themselves to the future integration of hardware within my system.

An ability to self-manage a significant piece of work.

Throughout the development of SAS, I have kept to the GANTT chart, *see Appendix A.1*. I originally designed in my project proposal, this has helped me keep on target, know how long I should roughly be spending on each piece of work, and accommodate more time when things have taken longer than first thought.

As well as the GANTT chart, I have been following an AGILE (Beck, 2001) style of development, aiming for focused, quick and intentional updates which I can present to my supervisor at meetings. I have been having supervisor meetings every fortnight during the development process, and have been treating these as “sprint reviews”, where I simply present the new additions to any code or documentation around my project, and receive instant feedback. This process has helped me keep on target, and ensured that any work I do undertake is worthwhile and will be reviewed before it gets too far. This helps prevent creating redundant features and getting distracted.

Critical self-evaluation of the process.

I think I approached this project well, and utilised my industry experience to help prepare, plan and execute the development of my software. I was able to perform activities and philosophies I had learnt throughout my university studies such as agile software development, development of Gantt charts and comprehensive software requirements. This planning process helped reduce the number of roadblocks encountered during implementation.

One of the strengths I showed during this project was my diverse technical knowledge. I recorded 7 languages used (C++, C#, CMake, Python, Batchfile, HTML, Bash) to create my system and multiple development environments and operating systems. Using languages such as Bash which is not taught heavily at university allowed me to create quick and powerful scripts that help automate my workflow and link different parts of the software together.

One of the parts of the software development that I have struggled with, would be the verification and validation of the software I have built. Because of the nature of the project, emulating the technical stack of a much larger and well defined digital twin system which a company might employ, I have found it difficult to validate my software, particularly things such as engine behaviours and network timings. During development I have been using sensible behaviour patterns when implementing my simulation of the engine, however, the ability to base the engine off a real piece of hardware, with clear documentation would be beneficial in making my system more well rounded.

9 References

- Alexander, D. E. (2017). *Nature's machines*. Academic Press.
- Alexandrescu, H. S. (2005). *C++ Coding Standards*. Stoughton, Massachusetts: John Wait.
- Beck, K. (2001). *Agile Manifesto*. Retrieved from Manifesto for Agile Software Development: <https://agilemanifesto.org/iso/en/manifesto.html>
- boost. (n.d.). *Boost*. Retrieved from Boost.Asio: https://www.boost.org/doc/libs/latest/doc/html/boost_asio.html
- Chen, G. L. (2024). *SFVInt: Simple, Fast and Generic Variable-Length Integer Decoding using Bit Manipulation Instructions*. ByteDance.
- Corrigan, S. (2016). *Introduction to the Controller Area Network (CAN)*. Texas Instruments.
- Docker. (n.d.). *Docker Desktop*. Retrieved from Docker: <https://docs.docker.com/desktop/>
- Fowler, M. (2003, March 05). *Model View Controller*. Retrieved from martinowler.com: <https://martinfowler.com/eaCatalog/modelViewController.html>

Gallagher, N. (2020, December 04). *What is a digital twin?* Retrieved from IBM: <https://www.ibm.com/think/topics/digital-twin>

Google. (n.d.). *googletest*. Retrieved from GitHub: <https://github.com/google/googletest>

Google. (n.d.). *Googletest*. Retrieved from GitHub: <https://google.github.io/googletest/>

Grives, M. (2016). *Origins of the Digital Twin Concept*. Florida: Florida Institute of Technology / NASA.

IOWN. (n.d.). *IOWN Global Forum Glossary*. Retrieved from IOWN Global Forum: <https://iowngf.org/glossary/#d>

J, P. (1980, August 28). *User Datagram Protocol*. Retrieved from RFC: <https://www.rfc-editor.org/rfc/rfc768>

JetBrains. (n.d.). *ReSharper*. Retrieved from jetbrains.com: <https://www.jetbrains.com/resharper/>

Martin, R. (2008). *Clean Code: A Handbook of Agile Software Craftsmanship*. Pearson.

Microsoft. (n.d.). *Microsoft*. Retrieved from .NET: <https://dotnet.microsoft.com/en-us/>

Microsoft. (n.d.). *Roslyn*. Retrieved from GitHub: <https://github.com/dotnet/roslyn>

MISRA. (2020). *MISRA Compliance:2020*. MISRA.

MIT. (n.d.). *nlohmann*. Retrieved from GitHub: <https://github.com/nlohmann/json>

Parasoft. (2011). *DO-178C Software Compliance for Aerospace & Defence*. Parasoft.

Prime Nearshore. (n.d.). *The Rise of Digital Twin – How is it Changing Tech?* Retrieved from Prime Nearshore: <https://www.primenearshore.com/insights/digital-twin-how-is-changing-tech>

Sandler, G. J. (2012). *The art of software testing*. New Jersey: John Wiley & Sons.

10 Appendices

Context:

Appendix A – Project planning

A.1 – SAS Project Plan GANTT Chart

Appendix B – System Verification

B.1 – Acceptance Tests

Appendix C – Communication Specification

C.1 – NEON Communications ICD

C.2 – Example WebSocket JSON

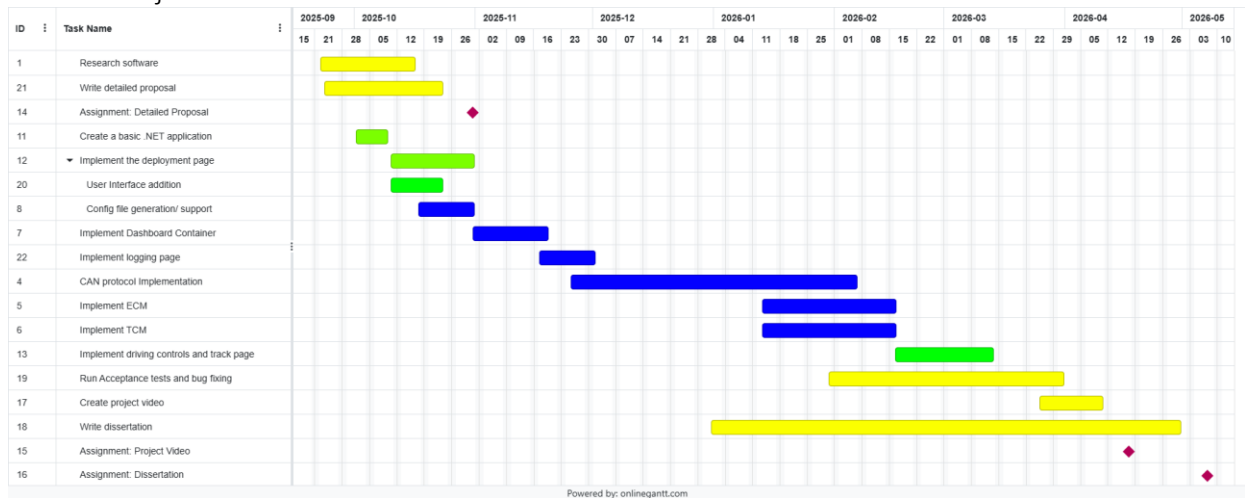
Appendix D – Configuration

D.1 – Example configuration file

Appendix E – Implementation Code

Appendix A – Project Planning

A.1 SAS Project Plan Gantt Chart



Appendix B – System Verification

B.1 Acceptance Tests

Acceptance tests

Prerequisites

- Windows is installed on the desired PC
- Docker desktop is running on the PC
- The code has been installed correctly

ID	Title	Steps	Expected Result	Pass/Fail
1	Opening the application	1.Open windows explorer and search SimulationPlatform 2. Press enter	Application opens	
2.	Closing the application	1. Open the application 2. Press the cross and close the application	Application closes gracefully	
3.	Application persistence after reopen	1.Open application 2.Close application 3.Reopen application	Application starts cleanly without crashes or corrupted state	
4.	Application minimisation	1.Open application 2.Minimise window	Application minimises and can be restored correctly	

Configuring & Deployment

Prerequisite

- The application has opened correctly, and the deployment page is open

ID	Title	Steps	Expected Result	Pass/Fail
----	-------	-------	-----------------	-----------

1	The user can choose different options for their NEON choice	1.Click the dropdown menus for each of the configurable components and select them	Selections are made and kept when moving through each dropdown.	
2	NEON cannot be deployed with an invalid selection	1.Do not assign at least one configurable	The deploy button is inactive	
3	NEON can be deployed with a valid selection	1. Assign a valid selection of configurable components. 2.Deploy the vehicle	The vehicle deploys and the application runs.	
4	NEON can be undeployed	1. Deploy NEON with a valid configuration 2.Un-deploy NEON	The un-deploy vehicle should be enabled and when pressed should disable and the Track and Logging page should disappear as it is undeployed.	
5	Deployment button feedback	1.Create valid configuration 2.Click deploy	Visual feedback, text saying "deploying..." is displayed	
6	Prevent duplicate deployment	1.Deploy NEON 2. Attempt to deploy again	Configuration drop-downs and deploy button are locked until undeployed	
7	Configuration persistence	1.Select configuration 2.Navigate away and return	Previously selected options remain saved	

Controlling and Monitoring NEON

Prerequisites

- The application has opened correctly
- A vehicle is correctly deployed
- The Track Page is open

ID	Title	Steps	Expected Result	Pass/Fail
1	The user is able to navigate to the track page	1.Click the track page icon on the left sidebar	The track page opens	
2	The user is able to navigate to the logging page	1.Click the logging page icon on the left sidebar	The logging page displays	
3	The track page is populated with data	1. Check the values are ensure the boxes are filled in on the track page	The track page data matches the logging page data	
4	The speed cannot be increased from neutral (gear 0)	1.Increase the acceleration	RPMs increase but speed does not	
5	Verification of acceleration	1.Get in gear 1 2.Increase throttle	The speed increases, RPMs increase	
6	Verification of brakes	1.Build up speed 2.Increase brake slider	The speed decreases, RPMs increase	

7	Verification of Warning	1.Drive vehicle beyond safe limits (e.g. high RPM)	Warning indicator activated	
8	Warning clears when safe	1.Trigger warning 2.Return to safe state	Warning indicator disappears	
9	Gear shifting validation	1.Increase gear sequentially	Gear changes correctly and reflects on the UI	
10	Data consistency between pages	1.Observe speed on track page 2. Compare with logging page	Values match in real-time	
11	Logging page updates continuously	1.Drive vehicle 2.Observe logging page	Logs update live without delay	
12	UI responsiveness under load	1.Rapidly adjust throttle/brake sliders	UI updates smoothly without freezing	
14	Vehicle stops completely	1.Apply full brakes at speed	Speed reduces to zero and remains stable	
15	System behaviour after undeploy	1.Undeploy vehicle 2.Try and reach the track or logging page	The track and logging page become inactive and cannot be reached on the VSP/ not visible	
16	Open the live graphs display	1.Click live graphs button	Live graphs window pops up and shows updating graphs	
17	Close live graphs display	1.Open the live graphs display 2.Close the live graphs display	Closes gracefully and returns to the track page	
18	Refuel button functionality	1.Press refuel	Fuel is refilled to 100%	

Appendix C – Communication Specification

C.1 Interface Control Document (ICD)

NEON ICD

Error Message

Purpose

An error message is a particular message that only the dashboard subscribes to, this is to indicate to the user interface that something is wrong or should be recommended and includes a plain text string to show this to the user.

Structure

Data field	Data type	Possible values	Description
Category	enum	Control = 1, Status = 2, Error = 3	Shows the message category
Type	uint8_t	Low Fuel =1, No Fuel =2, Engine Overheating =3, Engine Stalled = 4,	The type of message it is sending.

		Shift Down Recommended = 5, Shift Up Recommended = 6	
Message	String	..	Gives a plain text description describing the error, to be shown to the user, can be used for debugging.

Command Message

Purpose

Control a specific function around the network.

Structure

Data field	Data type	Possible values	Description
Category	uint8_t	Control = 1, Status = 2, Error = 3	Shows the message category
Type	uint8_t	GearUpRequest = 1, GearDownRequest = 2 Refuel = 3, ThrottleRequest = 4, BrakeRequest = 5	Gives an uint8_t value for the request.

Status Message

Purpose

Give the status of a particular part of the system periodically around the network. These are subscribed to across the network and are used in other components equations.

Structure

Data field	Data type	Possible values	Description
Category	uint8_t	Control = 1, Status = 2, Error = 3	Shows the message category
Type	uint8_t	Current Gear = 1, Acceleration = 2, Engine Temperature = 3, Fuel = 4, RPM = 5, Empty Fuel = 6, Speed = 7	Gives a value indicating the type of the message

Value	uint32	0...2,147,483,647	Gives a plain text description describing the error, to be shown to the user, can be used for debugging.
--------------	--------	-------------------	--

Speed Control

Purpose

Message with the intent to change the speed of the vehicle.

Structure

Data field	Data type	Possible values	Description
Category	uint8_t	Control = 1, Status = 2, Error = 3	Shows the message category
Type	uint8_t	GearUpRequest = 1, GearDownRequest = 2 Refuel = 3, ThrottleRequest = 4, BrakeRequest = 5	Gives the message type.
Value	uint32	0...2,147,483,647	Gives the numerical value parsed into the message.

C.2 Example WebSocket JSON

```
{
  "speed": 0,
  "gear": 1,
  "rpms": 0,
  "maxRpms": 6000,
  "engineTemp": 22,
  "fuel": 0,
  "errors": [
    {
      "code": 1,
      "msg": "Critical: Empty fuel."
    },
    {
      "code": 3,
      "msg": "Critical: Engine has stalled due to over-rev."
    }
  ],
  "status": true
}
```

Appendix D – Configuration

D.1 Example configuration file

```
{
  "engine": {
    "id": "engine_1L",
    "displacement_l": 1,
    "idle_rpm": 850,
    "max_rpm": 6000,
    "max_torque_nm": 110
  },
  "transmission": {
    "id": "transmission_5spd",
    "gears": 5,
    "final_drive": 3.9,
    "gear_ratios": [
      3.91,
      2.14,
      1.36,
      1.03,
      0.84
    ],
    "reverse_ratio": -3.54
  }
}
```

Appendix E – Implementation Code

E.1 Engine source code

```
#include "Engine.h"

#include <algorithm>
#include <chrono>
#include <cmath>

namespace ecm::engine
{
    using clock_t = std::chrono::steady_clock;

    shared::config::Engine Engine::sanitizeEngineCfg(shared::config::Engine
cfg)
    {
        cfg.idle_rpm = std::max(cfg.idle_rpm, 500);
        cfg.max_rpm = std::max(cfg.max_rpm, 1500);
        if (cfg.idle_rpm >= cfg.max_rpm)
        {
            cfg.idle_rpm = std::max(500, cfg.max_rpm - 1000);
        }

        cfg.max_torque_nm = std::max(cfg.max_torque_nm, 10.0);

        cfg.displacement_l = std::max(cfg.displacement_l, 0.1);
        return cfg;
    }

    shared::config::Transmission
Engine::sanitizeTransCfg(shared::config::Transmission cfg)
    {
        cfg.m_gears = std::max(cfg.m_gears, 0);

        cfg.m_finalDrive = std::max(cfg.m_finalDrive, 0.01);

        // Ensure gearRatios length matches gears if possible
        if (cfg.m_gears > 0 && static_cast<int>(cfg.m_gearRatios.size()) <
cfg.m_gears)
        {
            // If config says 5 gears but only gives 4 ratios, clamp gears
to provided ratios
            cfg.m_gears = static_cast<int>(cfg.m_gearRatios.size());
        }

        if (std::abs(cfg.m_reverseRatio) < 0.01)
            cfg.m_reverseRatio = -3.0;

        return cfg;
    }

    Engine::Engine(const shared::config::Engine& engineCfg,
        const shared::config::Transmission& transCfg)
        : m_engineCfg(sanitizeEngineCfg(engineCfg))
        , m_transCfg(sanitizeTransCfg(transCfg))
        , m_finalDrive(m_transCfg.m_finalDrive)
    {
```

```

        m_selectedGear = 0;
        m_gearRatio = 0.0;
        m_fuel.initialize(m_engineCfg.displacement_l);

        // Initialize RPM at idle
        m_rpm = static_cast<double>(m_engineCfg.idle_rpm);
    }

Engine::~~Engine()
{
    stop();
}

void Engine::start()
{
    std::scoped_lock lk(m_mutex);
    if (m_running)
        return;

    m_stopRequested = false;
    m_running = true;
    m_thread = std::thread(&Engine::runLoop, this);
}

void Engine::stop()
{
    {
        std::lock_guard<std::mutex> lk(m_mutex);
        if (!m_running)
        {
            return;
        }

        m_stopRequested = true;
    }
    m_cv.notify_all();

    if (m_thread.joinable())
        m_thread.join();

    {
        std::lock_guard<std::mutex> lk(m_mutex);
        m_running = false;
        m_stopRequested = false;
    }
}

bool Engine::isRunning() const
{
    std::lock_guard<std::mutex> lk(m_mutex);
    return m_running;
}

bool Engine::isStalled() const
{
    std::lock_guard<std::mutex> lk(m_mutex);
    return m_isStalled;
}

```

```

}

double Engine::getFuelPercentage() const
{
    std::lock_guard<std::mutex> lk(m_mutex);
    return m_fuel.getFuelPercent();
}

bool Engine::isOutOfFuel() const
{
    std::lock_guard<std::mutex> lk(m_mutex);
    return m_fuel.isEmpty();
}

void Engine::refuel()
{
    std::lock_guard<std::mutex> lk(m_mutex);
    m_fuel.refuel();
}

void Engine::setThrottle(std::uint32_t throttlePercent)
{
    {
        std::lock_guard<std::mutex> lk(m_mutex);
        m_throttlePercent = (throttlePercent > 100u) ? 100u :
throttlePercent;
    }
    m_cv.notify_all();
}

void Engine::setBrakeLevel(std::uint32_t brakePercent)
{
    {
        std::lock_guard<std::mutex> lk(m_mutex);
        m_brakePercent = (brakePercent > 100u) ? 100u : brakePercent;
    }
    m_cv.notify_all();
}

void Engine::setSelectedGear(int gear)
{
    std::lock_guard<std::mutex> lk(m_mutex);

    m_selectedGear = gear;

    // Neutral
    if (gear == 0)
    {
        m_gearRatio = 0.0;
        m_cv.notify_all();
        return;
    }

    // Reverse
    if (gear == -1)
    {
        m_gearRatio = m_transCfg.m_reverseRatio;
    }
}

```



```

        m_cv.notify_all();
        return;
    }

    // Forward gears 1..N (prefer ratio vector length)
    const int n = static_cast<int>(m_transCfg.m_gearRatios.size());
    if (gear >= 1 && gear <= n)
    {
        m_gearRatio = m_transCfg.m_gearRatios[gear - 1];
    }
    else
    {
        // Invalid gear -> neutral
        m_selectedGear = 0;
        m_gearRatio = 0.0;
    }

    if (m_gearRatio < 0.01)
        m_gearRatio = 0.0;

    m_cv.notify_all();
}

std::uint32_t Engine::getSpeedMph() const
{
    std::lock_guard<std::mutex> lk(m_mutex);
    const double mph = mpsToMph(m_speedMps);
    return static_cast<std::uint32_t>(std::round(std::max(0.0, mph)));
}

std::uint32_t Engine::getRpm() const
{
    std::lock_guard<std::mutex> lk(m_mutex);
    return static_cast<std::uint32_t>(std::round(std::max(0.0,
m_rpm))));
}

double Engine::getAccelerationMps2() const
{
    std::lock_guard<std::mutex> lk(m_mutex);
    return m_accelMps2;
}

double Engine::getCoolantTempC() const
{
    std::lock_guard<std::mutex> lk(m_mutex);
    return m_thermal.getCoolantTempC();
}

bool Engine::isOverheating() const
{
    std::lock_guard<std::mutex> lk(m_mutex);
    return m_thermal.isOverheating();
}

double Engine::clampd(double v, double lo, double hi)

```

```

{
    return std::max(lo, std::min(v, hi));
}

double Engine::mpsToMph(double mps)
{
    return mps / 0.44704;
}

// Config-driven smooth curve:
// - peak around ~60% of (redline-idle) above idle
// - falls toward redline
// - minimum floor near idle
double Engine::torqueAtRpmNm(const double rpm) const
{
    constexpr double kMinTorqueNm = 1.0;
    constexpr double kPeakRpmFraction = 0.60; // peak torque at 60% of
the RPM range
    constexpr double kSpreadFraction = 0.22; // gaussian width as
fraction of RPM range
    constexpr double kMinTorqueFraction = 0.25; // torque floor as
fraction of peak

    const auto maxT = std::max(kMinTorqueNm,
m_engineCfg.max_torque_nm);
    const auto redline = m_engineCfg.max_rpm;
    const auto idle = m_engineCfg.idle_rpm;

    if (redline <= idle + 1.0)
        return maxT;

    const auto rpmRange = redline - idle;
    const auto peakRpm = idle + kPeakRpmFraction * rpmRange;
    const auto spread = std::max(1.0, kSpreadFraction * rpmRange);
    const auto x = (rpm - peakRpm) / spread;

    return std::max(maxT * std::exp(-0.5 * x * x), kMinTorqueFraction *
maxT);
}

double Engine::computeRedlineSpeedCapMps() const
{
    // No cap if neutral/invalid ratio
    const double gear = m_gearRatio;
    const double fd = m_finalDrive;

    if (gear <= 0.01 || fd <= 0.01)
        return -1.0;

    const double wheelCirc = 2.0 * 3.141592653589793 * m_wheelRadiusM;
    if (wheelCirc <= 1e-6)
        return -1.0;

    const double totalRatio = gear * fd;
    if (totalRatio <= 1e-6)
        return -1.0;
}

```

```

    const double redlineRpm = static_cast<double>(m_engineCfg.max_rpm);

    const double maxWheelRpm = redlineRpm / totalRatio;
    const double maxWheelRps = maxWheelRpm / 60.0;
    const double maxSpeedMps = maxWheelRps * wheelCirc;
    return maxSpeedMps;
}

void Engine::runLoop()
{
    const auto tickPeriod =
        std::chrono::duration<double>(1.0 / std::max(1.0, m_tickHz));

    auto last = clock_t::now();

    for (;;)
    {
        {
            std::unique_lock<std::mutex> lk(m_mutex);
            m_cv.wait_for(lk, tickPeriod, [&] { return m_stopRequested;

                if (m_stopRequested)
                    break;
            }

            const auto now = clock_t::now();
            std::chrono::duration<double> dt = now - last;
            last = now;

            const double dtSec = clampd(dt.count(), 0.0, 0.1);
            step(dtSec);
        }
    }

void Engine::step(double dtSeconds)
{
    std::lock_guard<std::mutex> lk(m_mutex);

    if (dtSeconds <= 0.0)
        return;

    m_fuel.update(
        m_rpm,
        m_effectiveThrottle,
        static_cast<double>(m_engineCfg.idle_rpm),
        static_cast<double>(m_engineCfg.max_rpm),
        dtSeconds);

    const double idleRpm = static_cast<double>(m_engineCfg.idle_rpm);
    const double redlineRpm = static_cast<double>(m_engineCfg.max_rpm);

    // If RPM exceeds redline by 10%, the engine blows/stalls
    const double overRevThreshold = redlineRpm * 1.10;
    if (m_rpm > overRevThreshold)
    {
        m_isStalled = true;
    }
}

```

```

    bool hasFuel = !m_fuel.isEmpty();
    bool canRun = hasFuel && !m_isStalled;

    // Engine only responds to throttle if it has fuel and isn't blown
    if (!canRun)
    {
        m_effectiveThrottle = 0.0;
    }
    else
    {
        const double targetThrottle =
static_cast<double>(m_throttlePercent) / 100.0;
        const double alpha = 1.0 - std::exp(-m_throttleResponse *
dtSeconds);
        m_effectiveThrottle = m_effectiveThrottle + (targetThrottle -
m_effectiveThrottle) * alpha;
    }

    m_effectiveThrottle = clampd(m_effectiveThrottle, 0.0, 1.0);

    const double targetBrake = static_cast<double>(m_brakePercent) /
100.0;
    const double brakeAlpha = 1.0 - std::exp(-10.0 * dtSeconds);
    m_effectiveBrake = m_effectiveBrake + (targetBrake -
m_effectiveBrake) * brakeAlpha;
    m_effectiveBrake = clampd(m_effectiveBrake, 0.0, 1.0);

    if (m_thermal.isOverheating())
    {
        m_effectiveThrottle *= 0.5;
    }

    const double resistForce = m_cRolling + (m_cDrag * m_speedMps *
m_speedMps);
    const double speedCapMps = computeRedlineSpeedCapMps();

    const double wheelCirc = 2.0 * 3.141592653589793 * m_wheelRadiusM;
    const double wheelRps = (wheelCirc > 1e-6) ? (m_speedMps /
wheelCirc) : 0.0;
    const double wheelRpm = wheelRps * 60.0;

    // Calculate brake force (Assuming ~1G maximum deceleration: mass *
9.81 m/s^2)
    const double maxBrakeForce = m_massKg * 9.81;
    double currentBrakeForce = maxBrakeForce * m_effectiveBrake;

    // Only apply heavy mechanical braking if the vehicle is actually
moving to prevent
    // massive negative acceleration values from building up while
stationary.
    if (m_speedMps < 0.01)
    {
        currentBrakeForce = 0.0;
    }

    // Neutral logic

```

```

        if (m_gearRatio <= 0.01)
        {
            const double targetNeutralRpm = canRun ? (idleRpm + (redlineRpm
- idleRpm) * (0.25 + 0.75 * m_effectiveThrottle)) : 0.0;
            m_rpm = m_rpm + (targetNeutralRpm - m_rpm) * (1.0 - std::exp(-
2.0 * dtSeconds));

            double netForce = -resistForce - currentBrakeForce;
            m_accelMps2 = netForce / std::max(1.0, m_massKg);
        }
        else
        {
            const double rpmFromSpeed = wheelRpm * m_gearRatio *
m_finalDrive;
            const double floorRpm = canRun ? idleRpm : 0.0;
            const double targetRpm = std::max(floorRpm, rpmFromSpeed);

            constexpr double kRpmSlewRate = 1800.0;
            const double maxDelta = kRpmSlewRate * dtSeconds;
            m_rpm += clampd(targetRpm - m_rpm, -maxDelta, maxDelta);

            const double generatingRpm = std::min(m_rpm, redlineRpm);
            const double engTorque = torqueAtRpmNm(generatingRpm) *
m_effectiveThrottle;
            const double wheelTorque = engTorque * m_gearRatio *
m_finalDrive * m_drivetrainEff;
            const double driveForce = (m_wheelRadiusM > 1e-6) ?
(wheelTorque / m_wheelRadiusM) : 0.0;

            double netForce = driveForce - resistForce - currentBrakeForce;

            // Engine Braking & Over-Rev Handling
            if (speedCapMps > 0.0 && m_speedMps > speedCapMps)
            {
                const double overSpeedMps = m_speedMps - speedCapMps;
                netForce -= (overSpeedMps * 500.0);

                netForce = std::min(netForce, 0.0);
            }
            else if (rpmFromSpeed > idleRpm && m_effectiveThrottle < 0.15)
            {
                // Ramps from 0 (at 15% throttle) to full (at 0% throttle)
                const double liftFraction = 1.0 - (m_effectiveThrottle /
0.15);

                const double engineBrakeFactor = liftFraction * 0.03;
                netForce -= (rpmFromSpeed * engineBrakeFactor);
            }
            // Severe mechanical drag if the engine is completely
dead/blown
            if (!canRun) {
                netForce -= 250.0;
            }

            m_accelMps2 = netForce / std::max(1.0, m_massKg);
        }
    }

```

```

        m_speedMps += m_accelMps2 * dtSeconds;
        m_speedMps = std::max(m_speedMps, 0.0);

        m_thermal.update(m_rpm, m_effectiveThrottle, idleRpm, redlineRpm,
dtSeconds);
    }
}

```

E.2 Fuel model source code

```

#include "FuelTank.h"
#include <algorithm>

namespace ecm::engine::models
{
    void FuelTank::initialize(const double displacementLitres)
    {
        m_displacementL = std::max(0.1, displacementLitres);
        // Scale tank capacity by engine size
        // 1.0L -> 45L tank
        // 2.0L -> 55L tank
        m_capacityL = 40.0 + (m_displacementL * 7.5);
        m_fuelLevelL = m_capacityL; // start full
    }

    void FuelTank::update(const double rpm,
        double throttle01,
        const double idleRpm,
        const double maxRpm,
        const double dtSeconds)
    {
        if (m_fuelLevelL <= 0.0)
            return;

        throttle01 = std::clamp(throttle01, 0.0, 1.0);
        const auto rpmNorm =
            std::clamp((rpm - idleRpm) / std::max(1.0, maxRpm - idleRpm),
0.0, 1.0);

        // Base idle consumption (L/sec)
        auto burnRate = 0.015 * m_displacementL;
        // Load component
        burnRate += 0.15 * m_displacementL * throttle01;
        // RPM influence
        burnRate += 0.25 * m_displacementL * rpmNorm * throttle01;
        // Rich mixture near redline
        if (rpmNorm > 0.85)
        {
            burnRate *= 1.15;
        }

        m_fuelLevelL -= burnRate * dtSeconds;
        m_fuelLevelL = std::max(m_fuelLevelL, 0.0);
    }

    bool FuelTank::isEmpty() const
    {

```

```

        return m_fuelLevelL <= 0.001;
    }

    double FuelTank::getFuelLitres() const
    {
        return m_fuelLevelL;
    }

    double FuelTank::getFuelPercent() const
    {
        if (m_capacityL <= 0.0)
            return 0.0;
        return (m_fuelLevelL / m_capacityL) * 100.0;
    }

    void FuelTank::refuel()
    {
        m_fuelLevelL = m_capacityL;
    }

    void FuelTank::addFuelLitres(const double litres)
    {
        if (litres <= 0.0)
            return;
        m_fuelLevelL += litres;
        m_fuelLevelL = std::min(m_fuelLevelL, m_capacityL);
    }
}

```

E.3 Test coverage report generation code

```

import os
import subprocess
import platform
import shutil
import argparse

# --- Configuration ---
BUILD_DIR = "build_coverage"
HTML_DIR = "coverage_html"

def run_command(cmd, cwd=None):
    print(f"\n>>> Running: {' '.join(cmd)}")
    result = subprocess.run(cmd, text=True, cwd=cwd)
    if result.returncode != 0:
        print(f"[-] Command failed with exit code {result.returncode}")
        exit(1)

def main():
    parser = argparse.ArgumentParser(description="Generate gcov coverage report for Neon shared library.")
    parser.add_argument("--rebuild", action="store_true", help="Force a clean rebuild.")
    parser.add_argument("--diagnose", action="store_true", help="Run gcovr with no filters to reveal raw paths.")
    args = parser.parse_args()

```

```

if platform.system() == "Windows":
    msys2_path = r"C:\msys64\ucrt64\bin"
    if not os.path.exists(msys2_path):
        print(f"[-] Error: Could not find MSYS2 at {msys2_path}")
        exit(1)
    os.environ["PATH"] = msys2_path + os.pathsep + os.environ["PATH"]

if args.rebuild:
    if os.path.exists(BUILD_DIR):
        print(f"[--rebuild] Cleaning: {BUILD_DIR}")
        shutil.rmtree(BUILD_DIR)
        cmake_cmd = ["cmake", "-B", BUILD_DIR, "-S", ".", "-DENABLE_COVERAGE=ON"]
        if platform.system() == "Windows":
            cmake_cmd.extend(["-G", "MinGW Makefiles"])
        run_command(cmake_cmd)
        run_command(["cmake", "--build", BUILD_DIR])
    else:
        if not os.path.exists(BUILD_DIR):
            print(f"[-] Build directory '{BUILD_DIR}' not found. Run with -rebuild first.")
            exit(1)
        print(f"[*] Skipping rebuild. Pass --rebuild to force a clean build.")

print("\n--- Running Unit Tests ---")
run_command(["ctest", "--output-on-failure"], cwd=BUILD_DIR)

root_abs = os.path.abspath(".").replace("\\", "/")
build_abs = os.path.abspath(BUILD_DIR).replace("\\", "/")

print("\n--- Generating HTML Coverage Report ---")
os.makedirs(HTML_DIR, exist_ok=True)
html_output = os.path.join(HTML_DIR, "neon_test_cov_report.html").replace("\\", "/")

if args.diagnose:
    subprocess.run([
        "gcovr",
        "--root", root_abs,
        "--object-directory", build_abs,
        "--verbose",
    ], text=True)
    return

html_cmd = [
    "gcovr",
    "--root", root_abs,
    "--object-directory", build_abs,

    "--filter", ".*shared.*",

    "--exclude", ".*tests/*",
    "--exclude", ".*\\\\\\\\tests\\\\\\\\.*", # catches backslash variant
    "--exclude", ".*tcm/*",
    "--exclude", ".*\\\\\\\\tcm\\\\\\\\.*",
    "--exclude", ".*ecm/*",

```



```

"--exclude", ".*\\\\\\ecm\\\\\\.*",
"--exclude", ".*\\/dashboard\\..*",
"--exclude", ".*\\\\\\dashboard\\\\\\.*",
"--exclude", ".*\\/abs\\..*",
"--exclude", ".*\\\\\\abs\\\\\\.*",
"--exclude", ".*\\/build_coverage\\..*",
"--exclude", ".*\\\\\\build_coverage\\\\\\.*",
"--exclude", ".*\\/deps\\..*",
"--exclude", ".*\\/vcpkg_installed\\..*",

"--exclude-unreachable-branches",
"--exclude-throw-branches",

"--html",
"--html-details",
"--html-title", "Neon Shared Library Coverage Report",
"-o", html_output,
]
run_command(html_cmd)

print(f"\n[+] Report at: {html_output}")
if platform.system() == "Windows":
    os.startfile(html_output.replace("/", "\\"))

if __name__ == "__main__":
    main()

```